

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Уральский государственный экономический университет»
(УрГЭУ)

КУРСОВАЯ РАБОТА

на тему: Разработка информационной системы
для подбора косметических средств

по междисциплинарному курсу МДК.02.01
Технология разработки программного обеспечения

профессионального модуля ПМ 02 ЭК Экзамен по модулю
(Осуществление интеграции программных модулей)

Колледж УрГЭУ
Специальность 09.02.07
Информационные системы
и программирование

Дата защиты: 21.10.2026
Оценка: _____

Студент Устюжанина Ю.В.
Группа 23-03 ИСиП

Руководитель:
Коломейцева В.С.,
преподаватель

Екатеринбург

2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ 6
1.1 АНАЛИЗ ОБЪЕКТА ИССЛЕДОВАНИЯ 6
1.1.1 Общая характеристика процесса выбора косметики 6
1.1.2 Характеристика субъектов и объектов системы 6
1.1.3 Основные проблемы и недостатки текущего процесса 7
1.2 СТРАТЕГИЯ ИНФОРМАТИЗАЦИИ ОБЪЕКТА ИССЛЕДОВАНИЯ 7
1.2.1 Обзор существующих решений 7
1.2.2 Цель и принципы стратегии 8
1.2.3 Механизм реализации стратегии 8
1.3 АНАЛИЗ ПРЕДМЕТА ИССЛЕДОВАНИЯ 9
1.3.1 Суть алгоритма подбора косметических средств 9
1.3.2 Методика расчёта рейтинга совместимости 10
1.3.3 Моделирование пользовательских сценариев 10
1.4 ПОСТАНОВКА ЗАДАЧИ НА АВТОМАТИЗАЦИЮ 11
1.4.1 Функциональные требования к системе 11
1.4.2 Нефункциональные требования 12
2 ПРОЕКТНАЯ ЧАСТЬ 13
2.1 ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОГО ОБЕСПЕЧЕНИЯ 13
2.1.1 Архитектура системы 13
2.1.2 Обоснование выбора СУБД 14
2.1.3 Логическая модель базы данных 15
2.1.4 Физическая реализация базы данных 17
2.1.5 Классификаторы и система кодирования информации 18
2.1.6 Организация информационной базы 20
2.1.7 Формы выходных документов 22
2.2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ 25
2.2.1 Структура программного обеспечения 25
2.2.2 Алгоритм подбора товаров 28
2.2.3 Логика обработки ответов пользователя 31
2.2.4 Моделирование пользовательских сценариев 32
2.3 УПРАВЛЕНИЕ ПРОЕКТОМ 32
2.3.1 Календарный план-график работ 32
2.3.2 Бюджет на разработку 34
2.3.3 Оценка рисков и способы их минимизации 34
3 РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ 36
3.1 ПРОГРАММНО-ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ РЕШЕНИЯ ПОСТАВЛЕННОЙ ЗАДАЧИ 36
3.1.1 Структура программного обеспечения 36
3.1.2 Спецификации программных модулей 36
3.1.3 Пользовательский интерфейс 37
3.1.4 Обеспечение информационной безопасности 41

3.2 ТЕСТИРОВАНИЕ, ОЦЕНКА КАЧЕСТВА И ВНЕДРЕНИЕ 43
3.2.1 Исходные данные и результаты работы системы 43
3.2.2 Поэтапное описание работы с системой 45
3.2.3 Порядок внедрения проекта 46
3.3 ОЦЕНКА ЭФФЕКТИВНОСТИ ПРОЕКТА 47
3.3.1 Анализ затрат на ресурсное обеспечение 47
3.3.2 Качественные и количественные эффекты 47
3.3.3 Расчёт экономической эффективности 48
ЗАКЛЮЧЕНИЕ 50
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 52
ПРИЛОЖЕНИЕ А 54
ПРИЛОЖЕНИЕ Б 55
ПРИЛОЖЕНИЕ В 56
ПРИЛОЖЕНИЕ Г 57
ПРИЛОЖЕНИЕ Д 58
ПРИЛОЖЕНИЕ Е 60

ВВЕДЕНИЕ

Тема моей курсовой работы – разработка информационной системы для подбора косметических средств. Я выбрала её, потому что при покупке косметики приходится одновременно учитывать назначение, состав, цену и личные предпочтения. В приложении я объединила условия выбора в анкету, на основе которой формируется подборка товаров.

Актуальность темы связана с автоматизацией сравнения товаров по нескольким условиям. Фильтры каталога позволяют сузить поиск, а рекомендательная система может дополнительно ранжировать результаты и объяснять, какие ответы пользователя повлияли на подбор.

Цель курсовой работы – разработать информационную систему, которая подбирает косметические средства на основе анкеты пользователя и алгоритма сравнения продуктов.

Задачи исследования:

– изучить, какие параметры влияют на выбор косметики, и собрать их в единую анкету;

– разработать правила сравнения товаров и расчёта рейтинга по ответам пользователя;

– спроектировать базу данных для хранения пользователей, анкет, продуктов и истории подборок;

– разработать веб-приложение на «C#» с использованием «ASP.NET Core»;

– реализовать ключевые функции: регистрацию, анкету, подбор, личный кабинет, корзину, избранное, оформление заказа и админ-панель;

– настроить «PWA-функционал», чтобы приложение можно было установить на телефон;

– протестировать основные функции системы, оценить затраты на разработку и возможные эффекты от её использования.

Объект исследования – процесс выбора и подбора косметики для ухода за кожей.

Предмет исследования – алгоритм подбора и ранжирования косметических средств по данным анкеты пользователя и характеристикам товаров.

При написании работы я использовала анализ научной и аналитической литературы, сравнительный анализ существующих решений, методы обработки данных, алгоритмическое моделирование, а также экспериментальное тестирование системы.

Персонализированный подбор косметики рассматривается в исследованиях рекомендательных систем. Например, в работе о гибридной системе рекомендаций объединены данные о товарах, отзывы и история предпочтений пользователя [8]. В своём проекте я выбрала подход на основе явно заданных правил: он не требует накопленной истории покупок и позволяет проследить вклад отдельных условий в рейтинг.

Самостоятельный результат работы – разработанный мной алгоритм начисления баллов и формирования пояснений к подборке. Значения весов заданы при разработке и отражают принятые приоритеты. Их оптимальность и влияние на качество рекомендаций требуют отдельной проверки.

Практическая значимость работы состоит в объединении анкетирования, сравнения товаров и сохранения подборок в одном приложении. Пользователь может повторить подбор после изменения анкеты, а администратор – обновить каталог через отдельный интерфейс.

В работе использованы данные о рынке косметики, аналитические обзоры, документация по «ASP.NET Core», «Entity Framework Core» и «PostgreSQL», а также материалы по «PWA» и веб-разработке.

Курсовая работа состоит из введения, трёх глав, заключения, списка использованных источников и приложений. В первой главе я рассматриваю предметную область и требования к системе. Во второй описываю базу данных, алгоритм и план разработки. В третьей представлены реализация приложения, результаты функциональных проверок и оценка затрат.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 АНАЛИЗ ОБЪЕКТА ИССЛЕДОВАНИЯ

1.1.1 Общая характеристика процесса выбора косметики

Я рассматриваю процесс выбора косметики для ухода за кожей. Он включает определение потребностей, поиск товаров, сравнение характеристик и принятие решения о покупке. Результат зависит как от данных о продукте, так и от условий, которые указал покупатель.

Отраслевые публикации показывают изменения в ассортименте и спросе на косметику [1–3]. Например, в обзоре Retail.ru со ссылкой на данные «Нильсен» сообщается, что в 2025 году продажи средств для лица в натуральном выражении сократились на 6,3% по сравнению с 2024 годом [3]. Эти данные характеризуют рынок, но сами по себе не доказывают эффективность конкретного алгоритма подбора.

В рамках проекта автоматизируются сбор предпочтений, расчёт рейтинга и сохранение результатов. Определение типа кожи остаётся за пользователем: приложение принимает его ответы и не проводит диагностику.

1.1.2 Характеристика субъектов и объектов системы

В процессе выбора и покупки косметики я выделяю несколько основных групп участников: покупатели, магазины косметики, а также консультанты и косметологи.

Покупатель задаёт условия поиска и выбирает товары из предложенной подборки. Магазин предоставляет сведения о товарах и обрабатывает заказ. Консультант или косметолог может помочь уточнить потребности, однако в моей системе рекомендации формируются по данным анкеты и каталога.

Объектами системы являются: анкета пользователя (15 параметров), продукт (29 характеристик), подборка (10 лучших товаров) и история подборок, сохраняемая в личном кабинете.

1.1.3 Основные проблемы и недостатки текущего процесса

Основная проблема для автоматизации – разрозненность данных. Пользователь сообщает предпочтения в одном месте, сравнивает товары в другом и отдельно сохраняет результаты. В приложении эти действия связаны: анкета используется при расчёте рейтинга, а полученная подборка записывается в историю.

Для решения этой задачи нужны единый формат описания товаров, правила их сравнения и возможность повторного подбора. Именно эти функции я положила в основу проекта.

1.2 СТРАТЕГИЯ ИНФОРМАТИЗАЦИИ ОБЪЕКТА ИССЛЕДОВАНИЯ

1.2.1 Обзор существующих решений

Сегодняшние решения для подбора косметики можно разделить на три типа. Первый – фильтры на маркетплейсах, второй – приложения с тестами, третий – профессиональные консультации.

В каталоге с фильтрами пользователь самостоятельно задаёт условия отбора и сравнивает оставшиеся товары. В сервисе с анкетой часть этой работы выполняет алгоритм: ответы преобразуются в правила отбора или рейтинг. При консультации специалист уточняет запрос в диалоге. Для своего приложения я выбрала анкетирование, поскольку ответы можно сохранять и повторно использовать при изменении каталога.

Исследования описывают разные подходы: подбор на основе характеристик кожи, глубокое обучение и гибридные рекомендации [4, 5, 7, 13]. Для сравнения с моим решением наиболее существенны требования к исходным данным и объяснимость результата. Гибридная система из работы [8] использует несколько источников предпочтений; в моём проекте

исходными данными служат анкета и карточки товаров. Такой подход проще для учебного приложения, но качество правил необходимо оценивать отдельно.

1.2.2 Цель и принципы стратегии

Предполагаемые пользователи приложения – покупатели, которым нужен подбор средств по нескольким условиям. Со стороны магазина с системой работает администратор: он заполняет и обновляет каталог. Спрос и возрастной состав аудитории в рамках проекта отдельно не исследовались.

Исходя из анализа существующих решений, я формулирую стратегию информатизации прикладных процессов:

Цель стратегии – связать сбор предпочтений, сравнение товаров и сохранение результатов в одном веб-приложении.

При проектировании я выделила следующие принципы: единообразие данных, понятные пояснения к рекомендациям и возможность обновления анкеты.

Для единообразия данных в анкете используются заранее заданные варианты ответов. Пояснения формируются из сработавших правил алгоритма. При изменении ответов пользователь может запустить подбор заново. Подробный состав параметров приведён в разделе 1.3, а их обработка – в разделе 2.2.

История позволяет вернуться к ранее рекомендованным товарам. Для работы с телефона предусмотрены адаптивная вёрстка и поддержка прогрессивного веб-приложения (PWA). Ожидаемая польза этих функций оценивается отдельно от факта их реализации.

1.2.3 Механизм реализации стратегии

Для реализации этой стратегии:

– разрабатываю базу данных на «PostgreSQL»;

– создаю алгоритм на «C#» с весовыми коэффициентами;

– проектирую веб-приложение на «ASP.NET Core»;

– настраиваю «PWA-функционал»;

– проверяю основные сценарии и оцениваю ограничения решения.

1.3 АНАЛИЗ ПРЕДМЕТА ИССЛЕДОВАНИЯ

1.3.1 Суть алгоритма подбора косметических средств

Предмет автоматизации – сравнение ответов пользователя с характеристиками товаров. Я описываю это сравнение через набор правил. Каждое правило меняет рейтинг и при необходимости добавляет пояснение или предупреждение.

Алгоритм получает на вход 15 параметров из анкеты пользователя. Условно их можно разделить на три группы: характеристики кожи, финансовые и сезонные предпочтения, а также дополнительные факторы.

К характеристикам кожи относятся: тип кожи (сухая, жирная, комбинированная, нормальная, чувствительная, обезвоженная, склонная к куперозу, не знаю); возраст (до 20, 20-30, 30-40, 40-50, 50+); проблемы (акне, пигментация, морщины, купероз, тусклый цвет, гиперчувствительность, чёрные точки, сухость); аллергии (парабены, отдушки, спирт, силиконы, сульфаты, глютен, орехи, эфирные масла); чувствительность к солнцу (да/нет); склонность к отёкам (да/нет); профессиональный уход (да/нет); предпочтение по текстуре (лёгкая/плотная).

К условиям выбора относятся бюджет, сезон, любимые бренды и цель ухода. В анкете предусмотрены бюджеты до 1000, 1000–3000, 3000–10000, 10000–25000 рублей и без ограничения. Дополнительно учитываются уровень стресса, тип питания и готовность к многоступенчатому уходу. Эти ответы обрабатываются как признаки для заданных правил, а не как медицинские показатели.

Рейтинг Score отражает сумму начисленных баллов и штрафов. Более высокое значение означает лучшее соответствие правилам программы; оно не

является вероятностью эффективности или безопасности косметического средства.

1.3.2 Методика расчёта рейтинга совместимости

Для расчёта рейтинга я задала приоритеты: совпадения типа кожи и заявленной проблемы имеют больший вес, чем текстура или сезон. Алгоритм также учитывает категорию товара и дополнительные бонусы. Подробные правила, числовые значения и пример расчёта приведены в разделе 2.2.2.

Веса выбраны мной при разработке. Они не заимствованы из клинической методики и не подтверждены сравнительным экспериментом. Ограничение такого подхода – возможность компенсации штрафа другими бонусами. Поэтому отсутствие конфликта по отдельному параметру нельзя гарантировать только высоким итоговым рейтингом.

1.3.3 Моделирование пользовательских сценариев

Я выделила три основных сценария: первичный подбор после регистрации, повторный подбор после изменения анкеты и возвращение к сохранённым результатам. В каждом случае используются данные текущего пользователя.

Анкета передаётся на сервер, после чего пользователь получает список товаров с пояснениями. Из результата можно перейти к карточке товара, добавить его в корзину или избранное. История формируется автоматически при выполнении подбора.

Взаимодействие пользователя и системы показано на диаграмме в нотации моделирования бизнес-процессов (BPMN) в разделе 2.2.4. Пошаговая работа с интерфейсом описана в разделе 3.2.2.

По сравнению с ручным поиском автоматизация сокращает число повторных действий: одна сохранённая анкета используется для сравнения со всем каталогом. При этом полнота и корректность характеристик товаров остаются ответственностью администратора.

1.4 ПОСТАНОВКА ЗАДАЧИ НА АВТОМАТИЗАЦИЮ

1.4.1 Функциональные требования к системе

Цель автоматизации – создать информационную систему, которая позволяет пользователю подбирать косметические средства на основе анкеты и алгоритма сравнения продуктов.

Проектируемый комплекс задач является частью информационной системы и включает в себя следующие функциональные блоки:

– управление пользователями (регистрация, авторизация, личный кабинет);

– сбор и хранение данных анкеты;

– алгоритмический расчёт рейтинга продуктов;

– формирование подборки рекомендаций;

– управление каталогом продуктов (админ-панель);

– работа с корзиной и избранным;

– поддержка PWA для мобильных устройств.

В основе системы лежит пошаговая анкета, после заполнения которой запускается алгоритм подбора. Всё взаимодействие строится по схеме: Пользователь – Анкета – Алгоритм – Подборка – Корзина/Избранное/История.

Я использую BPMN для разделения действий пользователя и системы. Схема основного сценария приведена в разделе 2.2.4, а порядок работы с интерфейсом – в разделе 3.2.2.

Каждая функция системы описана через входные данные, выходные данные и регламент выполнения. Спецификация функциональных требований приведена в таблице 1.

Таблица 1 – Спецификация функциональных требований[1]

Функция	Вход	Выход	Регламент
Регистрация	Имя, «email», пароль	Созданный аккаунт	Ввод данных – проверка на уникальность – сохранение в БД
Заполнение анкеты	Параметры	Сохранённый профиль	Пошаговый опрос – сохранение данных в БД
Личный кабинет	ID пользователя	Данные профиля, история подборок, избранное	Запрос к БД – отображение на странице
Подбор продуктов	Данные анкеты	Подборка продуктов с рейтингом	Алгоритм сравнивает анкету с каждым продуктом – вычисляет рейтинг – сортирует – выводит результат
Корзина	Список выбранных продуктов	Итоговая стоимость	Добавление/удаление – пересчёт суммы
Админ-панель	Данные о продукте	Добавление/редактирование/удаление продукта	Заполнение полей – сохранение – обновление каталога

1.4.2 Нефункциональные требования

Требования к программно-технической среде: C#, ASP.NET Core MVC, PostgreSQL, браузер с поддержкой JavaScript и защищённое соединение HTTPS. Для серверной части требуется среда выполнения ASP.NET Core. Публикация всего приложения на GitHub Pages не предусмотрена: сервис не исполняет серверный код .NET.

Для учебного каталога целевой срок формирования подборки – не более двух секунд. Это требование нуждается в измерении на выбранном сервере. Надёжность обеспечивается проверкой обязательных полей и обработкой пустого каталога. Требования безопасности включают хеширование паролей, ограничение административных действий и передачу данных по HTTPS. Интерфейс должен сохранять читаемость на экранах компьютера и телефона.

[1] Составлено автором

2 ПРОЕКТНАЯ ЧАСТЬ

2.1 ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОГО ОБЕСПЕЧЕНИЯ

2.1.1 Архитектура системы

Разрабатываемая система представляет собой веб-приложение на «ASP.NET Core MVC» с базой данных на «PostgreSQL». Выбор архитектурного паттерна «Model-View-Controller» обусловлен необходимостью чёткого разделения данных, бизнес-логики и отображения. Такое разделение упрощает разработку и поддержку: изменение интерфейса не затрагивает логику, а корректировка алгоритма не требует переписывания представлений [9].

Архитектура системы является классической клиент-серверной. Пользователь открывает сайт в браузере или в режиме «PWA», если приложение установлено на мобильное устройство. Браузер отправляет «HTTP-запрос» на сервер, сервер обрабатывает его через контроллеры, обращается к базе данных через «Entity Framework Core», получает необходимые данные и возвращает готовую страницу. Всю логику подбора я вынесла в отдельный сервис «RecommendationService», чтобы не дублировать код в контроллерах.

Система разделена на три уровня. Клиент – это браузер или «PWA» на мобильном устройстве, он отвечает за отображение данных и отправку запросов. Сервер работает на «ASP.NET Core MVC» и содержит контроллеры для обработки запросов, сервисы с бизнес-логикой и контекст базы данных для работы с данными. База данных – «PostgreSQL», хранит пользователей, анкеты, товары, подборки, заказы и избранное.

Схема взаимодействия уровней системы приведена на рисунке 1.

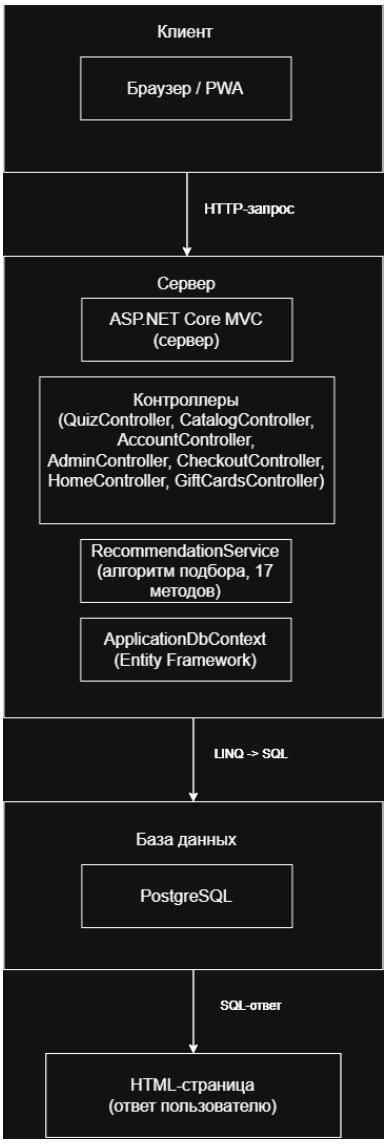


Рисунок 1 – Схема клиент-серверного взаимодействия[2]

2.1.2 Обоснование выбора СУБД

В качестве СУБД я выбрала PostgreSQL. Для проекта важны возможность работы без покупки лицензии, поддержка связей и ограничений целостности, а также интеграция с Entity Framework Core через провайдер Npgsql. По сравнению с файловой базой данных серверная СУБД требует отдельной настройки, но позволяет обслуживать запросы нескольких пользователей [10, 12].

Даты регистрации, заказов и подборок сохраняются в UTC. Для соответствующих полей используется тип timestamp with time zone. Это

[2] Составлено автором

позволяет хранить моменты времени единообразно; отображение в часовом поясе пользователя решается на уровне приложения [12].

Через «NuGet» установлены четыре ключевых пакета:

– «Npgsql.EntityFrameworkCore.PostgreSQL» (8.0.8): провайдер для «PostgreSQL»;

– «Microsoft.EntityFrameworkCore.Tools» (8.0.8): инструменты миграций;

– «Microsoft.EntityFrameworkCore.Design» (8.0.8): «design-time» компоненты;

– «BCrypt.Net-Next (4.2.0)»: библиотека хеширования паролей.

Версии пакетов указаны в файле проекта LuxDustApp.csproj. Базовые компоненты Entity Framework Core подключаются также через зависимости этих пакетов. Состав пакетов показан на рисунке 2.

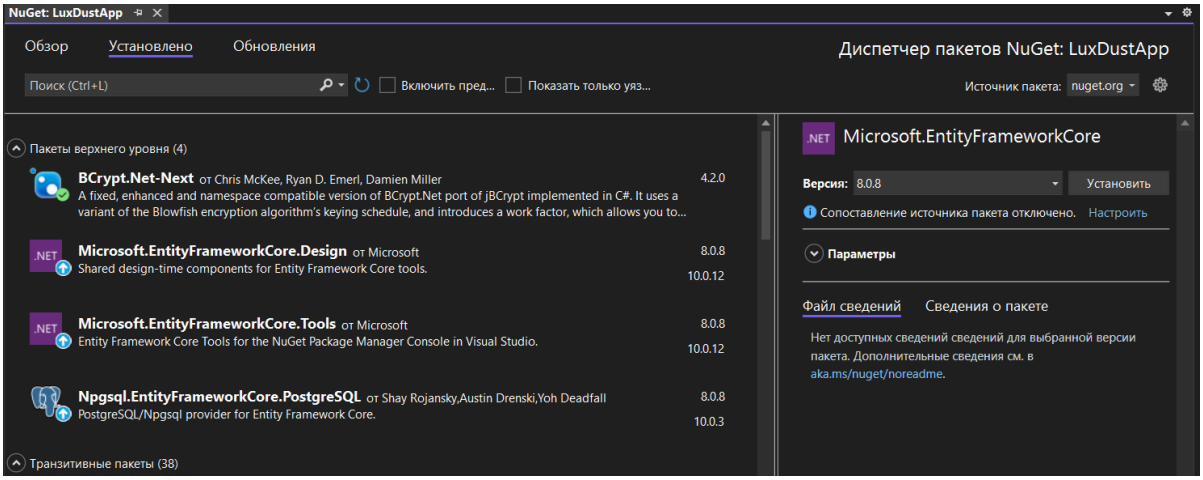


Рисунок 2 – Установленные «NuGet-пакеты»[3]

2.1.3 Логическая модель базы данных

База данных включает 11 таблиц. Краткое описание таблиц приведено в таблице 2.

[3] Составлено автором

Таблица 2 – Состав таблиц базы данных[4]

Таблица	Назначение
«Users»	Пользователи: «email», хеш пароля, имя, роль (админ), дата регистрации
«Profiles»	Анкеты: 15 полей (тип кожи, возраст, бюджет, проблемы, аллергии, сезон, бренды, цель, стресс, питание, солнце, отёки, проф. уход, текстура, многоступенчатый уход)
«Products»	Товары: название, бренд, категория, подкатегория, цена, тип кожи, проблема, сезон, описание, рейтинг, флаги «Новинка»/«Акция», 8 аллергенов, дополнительные параметры
«Recommendations»	История подборок: товар, пользователь, рейтинг, дата
«Favorites»	Избранное: пользователь, товар, дата добавления
«Carts»	Корзина: пользователь, товар, количество, дата
«PromoCodes»	Промокоды: код, процент скидки, фиксированная скидка, минимальная сумма, срок действия, лимит использований, счётчик использования
«GiftCards»	Подарочные карты: код, номинал, остаток, активность, владелец, дата покупки, дата использования, срок действия
«OrderGiftCards»	Связь заказов и подарочных карт: заказ, карта, сумма списания
«Orders»	Заказы: способ доставки, адрес, дата, время, телефон, сумма, статус
«OrderItems»	Товары в заказе: заказ, товар, количество, цена на момент заказа

Каждая анкета содержит внешний ключ UserId. В текущей логике приложения для пользователя создаётся одна анкета, а при повторном заполнении она обновляется. Само наличие внешнего ключа не обеспечивает отношение 1:1: для этого требуется уникальное ограничение. Таблицы рекомендаций, избранного и корзины связывают пользователя с товаром. Заказ связан с пользователем, а OrderItems хранит позиции заказа.

На схеме в приложении А показаны 11 таблиц, их первичные ключи и ссылки между сущностями. Неключевые поля описаны в таблице 2 и моделях проекта. Связь анкеты с пользователем на уровне приложения предполагает одну актуальную анкету; на уровне БД для этого требуется уникальное ограничение UserId.

[4] Составлено автором

2.1.4 Физическая реализация базы данных

Физическая реализация выполнена через миграции «Entity Framework Core». «SQL-скрипты» генерируются автоматически на основе «C#-моделей». При добавлении нового поля в модель я выполняю команду «Add-Migration», затем «Update-Database» – база обновляется без ручного написания «SQL» [10].

SQL-описание таблицы Products, приведённое в приложении Б, согласовано с типами текущей модели Product. Перед развёртыванием структуру всей базы необходимо проверить по актуальному набору миграций.

Модель Product содержит 29 сохраняемых свойств: Price имеет тип integer, Rating – double precision, CreatedAt – timestamp with time zone. Строки в текущем C#-классе допускают NULL, логические признаки представлены типом bool. На рисунке 3 показана структура базы на этапе разработки. Её ограничения NULL местами отличаются от текущей модели; перед внедрением нужно согласовать модель и миграции.

Data Output | Messages | Notifications

	column_name name	data_type character varying	is_nullable character varying (3)	column_default character varying
1	Id	integer	NO	[null]
2	Name	text	NO	[null]
3	Brand	text	NO	[null]
4	Category	text	NO	[null]
5	Price	integer	NO	[null]
6	SkinType	text	NO	[null]
7	Problem	text	NO	[null]
8	AllergenFree	boolean	NO	[null]
9	Season	text	NO	[null]
10	ImageUrl	text	NO	[null]
11	Description	text	NO	[null]
12	Rating	double precision	NO	[null]
13	CreatedAt	timestamp with time zo...	NO	[null]
14	SunSensitivity	boolean	YES	false
15	TendencyToEdema	boolean	YES	false
16	TexturePreference	text	YES	[null]
17	HasProfessionalC...	boolean	YES	false
18	ReadyForMultiStep	boolean	YES	false
19	SubCategory	text	YES	''::text
20	IsNew	boolean	YES	false
21	IsOnSale	boolean	YES	false
22	HasParabens	boolean	YES	false
23	HasFragrance	boolean	YES	false
24	HasAlcohol	boolean	YES	false
25	HasSilicones	boolean	YES	false
26	HasSulfates	boolean	YES	false
27	HasGluten	boolean	YES	false
28	HasNuts	boolean	YES	false
29	HasEssentialOils	boolean	YES	false

Рисунок 3 – Структура таблицы «Products» в «pgAdmin»[5]

В модели Profile строковые ответы допускают NULL. Для отсутствующих значений алгоритм использует проверки на пустую строку. Ключевые поля SkinType, Age и Budget дополнительно проверяются в контроллере перед подбором. Поля Name, Email и PasswordHash пользователя обязательны на уровне модели.

2.1.5 Классификаторы и система кодирования информации

В системе используется несколько классификаторов, обеспечивающих единообразие обработки данных. Структура кодовых обозначений объектов показана в таблице 3.

[5] Составлено автором

Таблица 3 – Представление справочных значений в приложении

Набор данных	Представление	Пример	Область применения
Тип кожи	Строка	Сухая	Анкета и товар
Возраст	Целое число	35	Анкета
Бюджет	Целое число	3000	Анкета
Проблемы кожи	Строка со списком	Акне, Пигментация	Анкета
Нежелательные компоненты	Строка и флаги boolean	Отдушки; HasFragrance	Анкета и товар
Сезон	Строка	Зима	Анкета и товар
Цель ухода	Строка со списком	Увлажнение	Анкета
Категория товара	Строка	Уход за лицом	Товар
Подкатегория	Строка	Уход за глазами	Товар
Статус заказа	Строка	Значение статуса	Заказ

Справочные значения используются внутри приложения и являются локальными. Тип кожи, сезон, цель и категория сохраняются строками. Возраст и бюджет представлены целыми числами, ответы «да/нет» – логическими значениями. Числовая иерархическая кодировка категорий в текущем коде не реализована; фактические способы представления сведены в таблицу 3.

Полный перечень классификаторов показан в таблице 4.

Таблица 4 – Перечень классификаторов и их значений[6]

Классификатор	Значения	Где используется
Тип кожи	Сухая, Жирная, Комбинированная, Нормальная, Чувствительная, Обезвоженная, Склонная к куперозу, Не знаю	«Profiles», «Products»
Возраст	До 20, 20-30, 30-40, 40-50, 50+	«Profiles»
Бюджет	До 1000, 1000-3000, 3000-10000, 10000-25000, Без лимита	«Profiles»
Проблемы	Акне, Пигментация, Морщины, Купероз, Тусклый цвет, Гиперчувствительность, Чёрные точки, Сухость, Расширенные поры, Отёчность и др.	«Profiles», «Products»
Аллергии	Парабены, Отдушки, Спирт, Силиконы, Сульфаты, Глютен, Орехи, Эфирные масла	«Profiles», «Products»
Сезон	Лето, Зима, Демисезон, Круглый год	«Profiles», «Products»
Цель ухода	Увлажнение, Питание, Антивозрастной, Очищение, Защита, Восстановление, Матирование, Лифтинг, Осветление, Сужение пор, Сияние, Снятие стресса	«Profiles», «Products»
Уровень стресса	Низкий, Средний, Высокий	«Profiles»
Тип питания	Сбалансированное, Сладкое/жирное, Вегетарианство	«Profiles»
Текстура	Лёгкая, Плотная	«Profiles», «Products»
Категории	17 категорий (Уход за лицом, Макияж, Волосы, Тело, Парфюмерия и др.)	«Products»
Подкатегории	Около 80 подкатегорий	«Products»

Администратор задаёт категорию, подкатегорию и характеристики при работе с товаром. Для сохранения истории при удалении товара требуется отдельно согласовать ограничения внешних ключей и поведение удаления. Физическое удаление записи само по себе не гарантирует сохранность связанных подборок и заказов.

2.1.6 Организация информационной базы

Информационная база включает PostgreSQL и файловое хранилище изображений. В ImageUrl сохраняется путь к изображению, а сами файлы

[6] Составлено автором

размещаются в wwwroot. Такое разделение позволяет обновлять файлы независимо от основных записей каталога.

Все данные делятся на две категории: нормативно-справочная информация и оперативная информация.

Нормативно-справочная информация (НСИ) – это «медленные» данные, которые редко изменяются: классификаторы (тип кожи, проблемы, аллергии, сезоны, цели ухода), категории и подкатегории товаров, справочная информация о брендах. Оперативная информация (ОИ) – это «быстрые» данные, которые изменяются ежедневно: пользователи и их анкеты, подборки, избранное, корзина, заказы и их статусы.

Каждая таблица базы данных имеет своё назначение, тип записи и регламент применения. Описание приведено в таблице 5.

Таблица 5 – Описание таблиц НСИ и ОИ[7]

Таблица	Тип	Назначение	Когда применяется
«Users»	ОИ	Хранение учётных записей пользователей и администраторов	При регистрации, входе, оформлении заказа
«Profiles»	ОИ	Хранение анкет пользователей с 15 параметрами	При заполнении/обновлении анкеты, при подборе
«Products»	НСИ	Хранение каталога косметических средств	При подборе, в каталоге, в корзине, в заказах
«Recommendations»	ОИ	История всех подборок пользователя	При каждом подборе, при просмотре ЛК
«Favorites»	ОИ	Избранные товары пользователя	При добавлении/удалении из избранного
«Carts»	ОИ	Текущая корзина пользователя	При добавлении/удалении товаров, при оформлении заказа
«PromoCodes»	НСИ	Промокоды для скидок	При оформлении заказа
«GiftCards»	ОИ	Подарочные карты пользователей	При покупке и применении карты
«OrderGiftCards»	ОИ	Связь заказов и карт	При оформлении заказа с картой
«Orders»	ОИ	Оформленные заказы	При оформлении заказа, при просмотре истории
«OrderItems»	ОИ	Товары в составе заказа	При оформлении и просмотре заказов

[7] Составлено автором

Администратор обновляет каталог через админ-панель – изменения автоматически отражаются в подборе.

2.1.7 Формы выходных документов

В системе предусмотрены несколько типов выходных форм, они адаптированы под мобильные устройства и работают в «PWA-режиме».

Справочные формы предназначены для просмотра данных. Каталог товаров, представленный на рисунке 4, содержит карточки с фото, названием, брендом, ценой и рейтингом; предусмотрена фильтрация по категориям, подкатегориям и цене, а также поиск по названию. Запросные формы формируются по запросу пользователя: страница подборки на рисунке 5 отображает ТОП-10 товаров с рейтингом совместимости и пояснениями, а также блок «Также может вам подойти».

Регламентированные формы формируются автоматически: личный кабинет на рисунке 6 содержит историю подборок с датами, раздел «Избранное» и данные анкеты. Контрольные формы предназначены для администратора: админ-панель на рисунке 7 отображает статистику системы и таблицу товаров с возможностью редактирования и удаления.

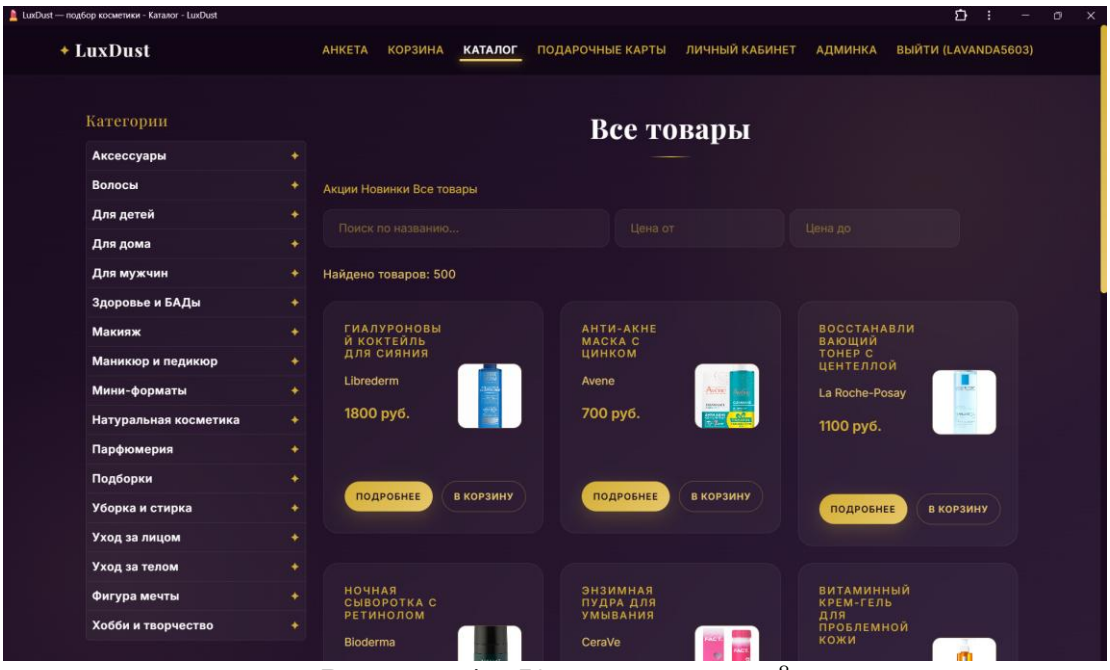


Рисунок 4 – Каталог товаров[8]

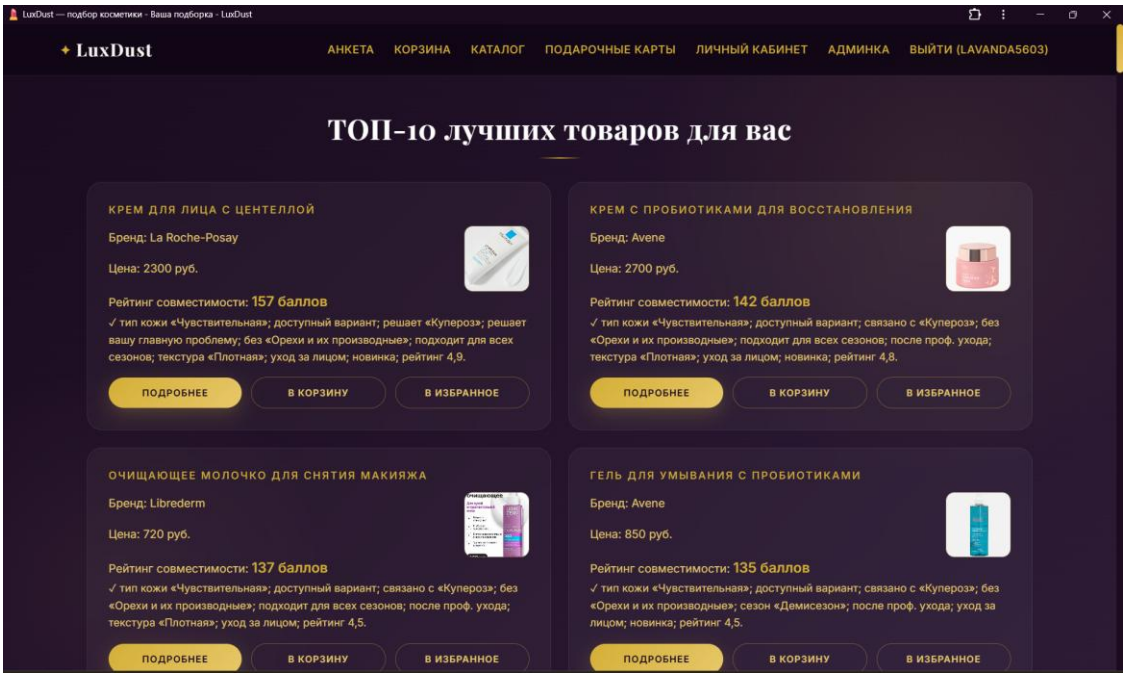


Рисунок 5 – Страница подборки[9]

[8] Составлено автором
[9] Составлено автором

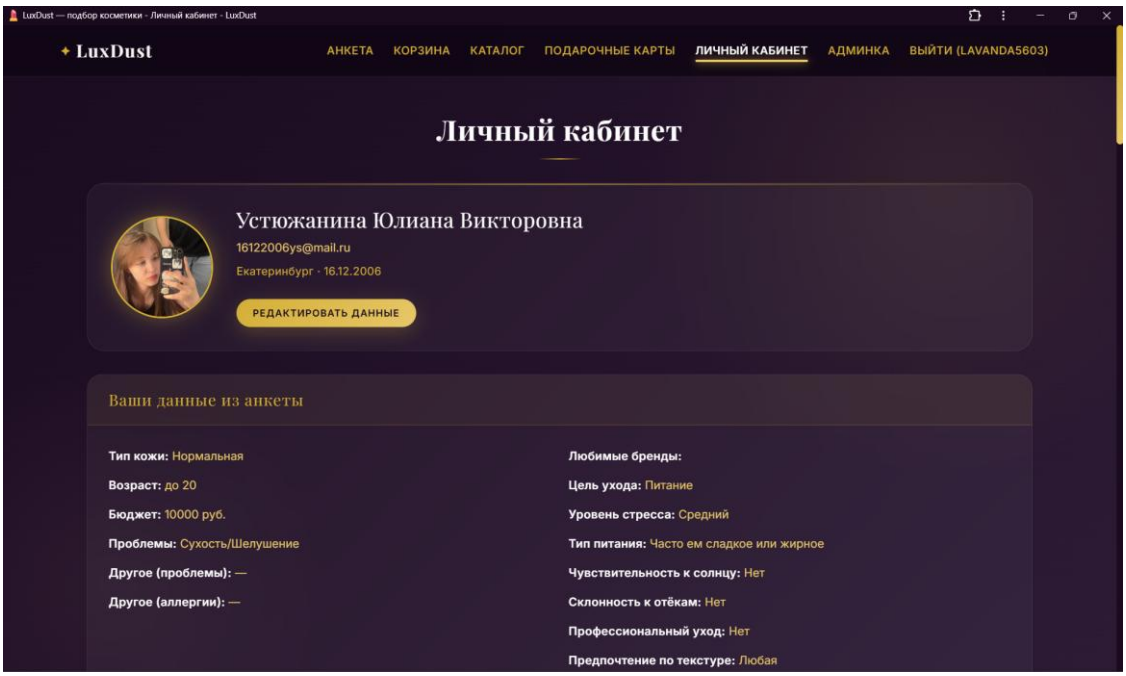


Рисунок 6 – Личный кабинет[10]

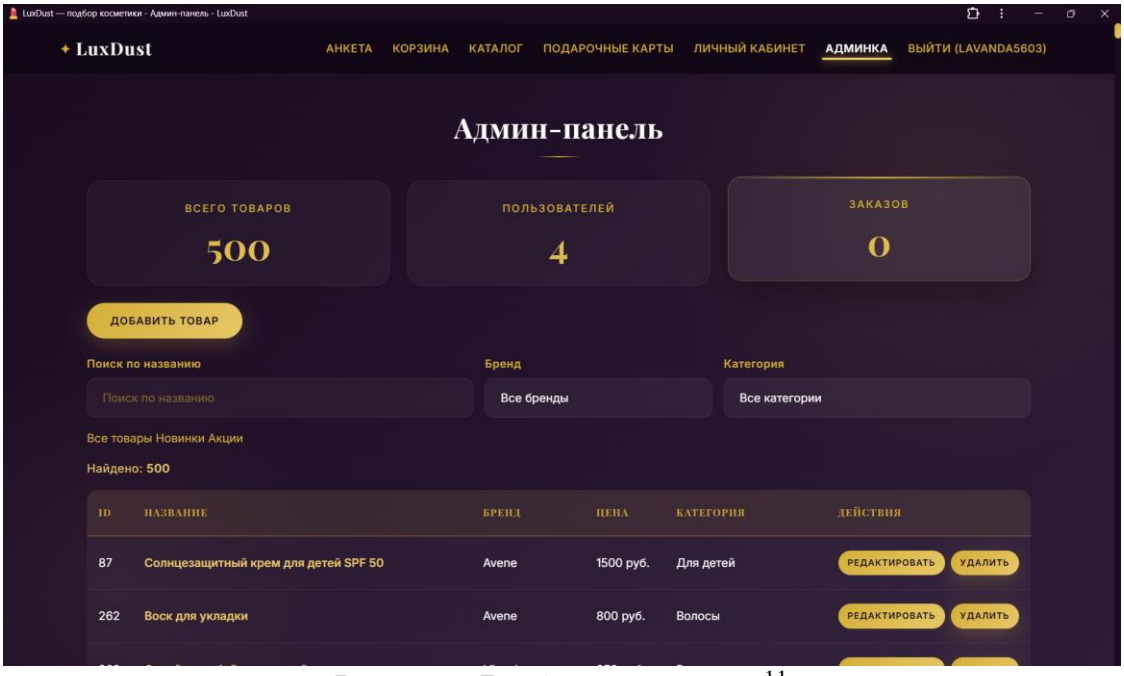


Рисунок 7 – Админ-панель[11]

[10] Составлено автором
[11] Составлено автором

2.2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.2.1 Структура программного обеспечения

Я разделила код по назначению, чтобы избежать дублирования и упростить поддержку. Папка Controllers содержит семь контроллеров: HomeController, QuizController, AccountController, CatalogController, CheckoutController, AdminController и GiftCardsController. В Models размещены сущности базы данных, модели представлений и классы результатов подбора.

Папка Services содержит RecommendationService с логикой подбора, RecommendationConstants с общими правилами и GiftCardService для работы с подарочными картами. Доступ к базе данных организован через ApplicationDbContext из папки Data.

Папка Views содержит представления Razor, сгруппированные по контроллерам. В wwwroot размещены стили, JavaScript, изображения и файлы PWA. Представления MVC в этом проекте не следует смешивать с отдельной технологией Razor Pages.

Структура проекта показана на рисунке 8.

«ViewModel» вынесены отдельно: «RegisterViewModel» – для валидации регистрации, «AdminProductViewModel» – для списков брендов и категорий в админке.

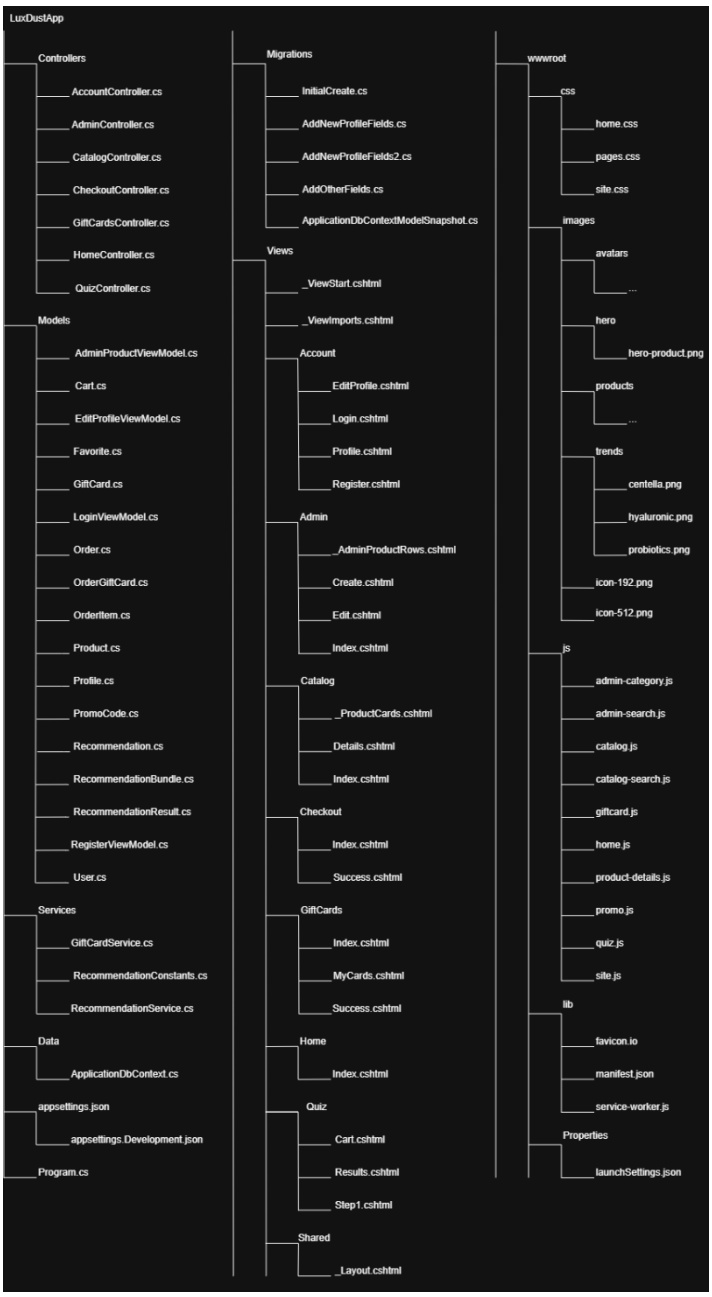


Рисунок 8 – Дерево программных модулей[12]

Для описания структуры программного обеспечения я использую схему классов на рисунке 9. Она показывает модели, сервис, контекст базы данных и контроллер, а также зависимости между ними.

Модели данных («User», «Profile», «Product») описывают сущности предметной области. Класс «User» содержит информацию о пользователе. Класс «Profile» хранит 15 параметров анкеты. Класс «Product» описывает косметическое средство.

[12] Составлено автором

RecommendationService выполняет расчёт рейтинга. Главный метод GetRecommendationsWithReasons() вызывает 17 проверок Check*(); кроме них в классе есть вспомогательный метод SolvesAnyProblem(). Подробный порядок вызовов рассмотрен в разделе 2.2.3.

Контекст базы данных «ApplicationDbContext» наследуется от «DbContext» и содержит по одному свойству «DbSet<T>» для каждой модели [10].

Контроллер «QuizController» обрабатывает «HTTP-запросы», связанные с анкетой и подбором.

Контроллер зависит от сервиса и контекста базы данных. Сервис использует контекст для загрузки товаров. Связи моделей задаются внешними ключами, а их фактическая кратность определяется ограничениями базы данных, рассмотренными в разделе 2.1.3.

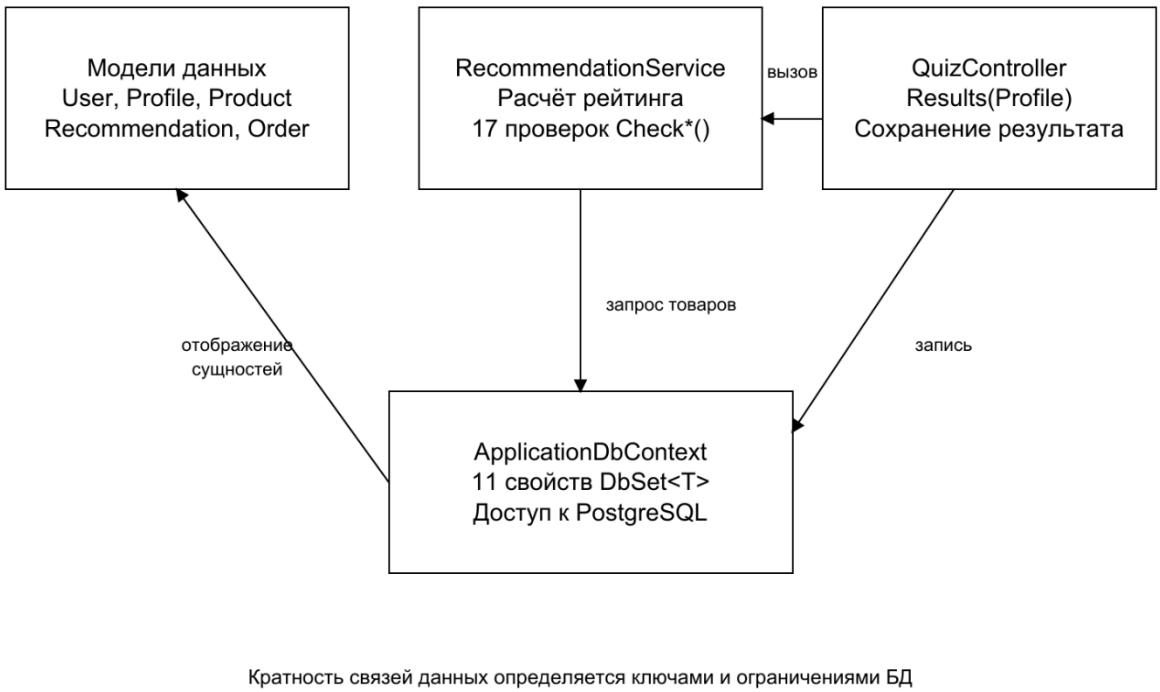


Рисунок 9 – «UML-диаграмма» классов[13]

[13] Составлено автором

2.2.2 Алгоритм подбора товаров

Алгоритм получает анкету и загружает товары из базы данных. Для каждого допустимого по категории товара он вычисляет рейтинг по формуле (1). В расчёте выделены 17 групп правил: 15 параметров анкеты, категория и бонусы товара.

Формула рейтинга:

$$\text{Score} = \sum_{i=1}^{17} w_i * f_i\,(\text{profile}, \text{product}), \qquad (1)$$

где Score – итоговый рейтинг; wi – масштаб баллов для i-й группы правил; fi(profile, product) – вклад совпадений и конфликтов в этой группе; profile – анкета; product – товар. Функция fi может учитывать несколько условий, поэтому её значение не ограничивается нулём и единицей. Формула описывает суммирование вкладов; в коде баллы изменяются непосредственно в методах Check*().

Веса факторов приведены в таблице 6.

Таблица 6 – Правила начисления баллов

№	Фактор	Вес (баллы)	Логика
1	Тип кожи	+40	Точное совпадение
—	Связанные типы кожи	+15…+25	Обезвоженная – Сухая, Купероз – Чувствительная
2	Бюджет	Бонус до +30; штраф до −100	Зависит от выбранного бюджета и превышения цены
3	Проблема	+30 за прямое; +15 за связанное	Дополнительные бонусы +10, +20, +30 по условиям совпадений
4	Аллергии	−50 за совпадение; +10 за отсутствие	При двух и более совпадениях ещё −100; фактического исключения нет
5	Сезон	+15 / +12 / +8 / −20	Совпадение, всесезонный товар, всесезонный запрос или конфликт
6	Любимые бренды	+25	Совпадение бренда
7	Цель ухода	+20	Совпадение цели
8	Возраст	+10…+30	5 групп с разными баллами
9	Уровень стресса	+5…+15	Высокий стресс – успокаивающие средства
10	Тип питания	+12…+15	Вегетарианство – веган-косметика
11	Чувствительность к солнцу	+20	«SPF-защита»
12	Склонность к отёкам	+10…+20	Средства от отёков
13	Профессиональный уход	-10 / +10	Штраф за скрабы
14	Текстура	+10	Совпадение текстуры
15	Многоступенчатый уход	+10…+15 / -50	Бонус за сыворотки, штраф при неготовности
16	Бонусы	+5 за каждый признак	Новинка, акция, рейтинг не ниже 4,5
17	Категория	+15 / −10	Дополнительный приоритет основной категории ухода

Блок-схема алгоритма приведена на рисунке 10.

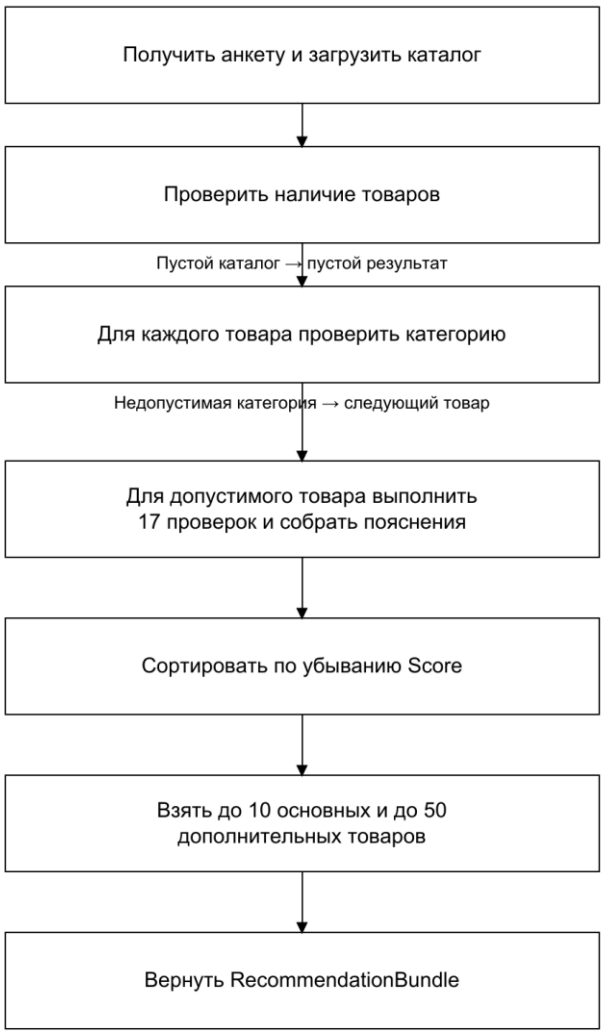


Рисунок 10 – Блок-схема алгоритма подбора[14]

Сначала алгоритм исключает служебные и неподходящие по запросу категории. Затем для каждого оставшегося товара выполняются проверки и собираются пояснения. После сортировки по Score первые десять товаров включаются в основную подборку, следующие пятьдесят – в дополнительный список. Если товаров меньше, возвращается доступное количество.

Пример частичного расчёта. Пользователь указал сухую кожу, возраст 35 лет, бюджет 3000 рублей, единственную проблему «Сухость/Шелушение», аллергию на отдушки и сезон «Зима». Рассматривается условный товар с типом кожи «Сухая», ценой 2300 рублей, проблемой «Сухость/Шелушение», сезоном «Круглый год» и без отдушек. Вклад пяти проверок:

[14] Составлено автором

– совпадение типа кожи: +40;

– бюджет: 2300 руб. попадает в диапазон: +30;

– совпадение единственной проблемы: +30 и дополнительный бонус +10;

– отсутствие указанного нежелательного компонента: +10;

– сезон: «Круглый год»: +12;

– сумма по этим пяти проверкам: 40 + 30 + 40 + 10 + 12 = 132 балла.

Это промежуточная сумма, а не окончательный рейтинг. Остальные проверки могут её изменить. Место товара в подборке определяется только после сравнения итоговых рейтингов всех допустимых товаров.

2.2.3 Логика обработки ответов пользователя

Обработка параметров распределена между методами в «RecommendationService». Всего таких методов 17: «CheckSkinType()», «CheckAge()», «CheckBudget()», «CheckProblems()», «CheckAllergies()», «CheckSeason()», «CheckBrands()», «CheckGoal()», «CheckStress()», «CheckDiet()», «CheckSunSensitivity()», «CheckEdema()», «CheckProfessionalCare()», «CheckTexture()», «CheckMultiStep()», «CheckCategory()», «CheckBonuses()».

Я разбила логику на методы, чтобы избежать дублирования и упростить поддержку. Пример обработки аллергий показан в приложении В.

Подход через словари позволил убрать длинные цепочки if-else. Проверка категории выполняется до начисления баллов и дополнительно в CheckCategory(). Проверка нежелательных компонентов устроена иначе: за совпадение начисляется штраф, но товар остаётся в списке. При двух и более совпадениях применяется ещё один штраф −100. Поэтому даже сообщение об исключении в текущем коде не означает фактическое удаление товара. Для строгого запрета таких рекомендаций необходим отдельный фильтр до сортировки.

2.2.4 Моделирование пользовательских сценариев

Основной пользовательский сценарий представлен на рисунке 11 в виде схемы с элементами BPMN. Две дорожки разделяют действия пользователя и системы.

Диаграмма показывает путь от ввода анкеты до просмотра результата и оформления заказа. Расчёт выполняет система, выбор товара и подтверждение заказа – пользователь. Сохранение рекомендаций происходит автоматически в контроллере, а не по отдельному запросу пользователя.

На схеме кругами обозначены начало и конец сценария, прямоугольниками – действия. Схема показывает основной успешный путь; обработка ошибок рассматривается отдельно при тестировании.

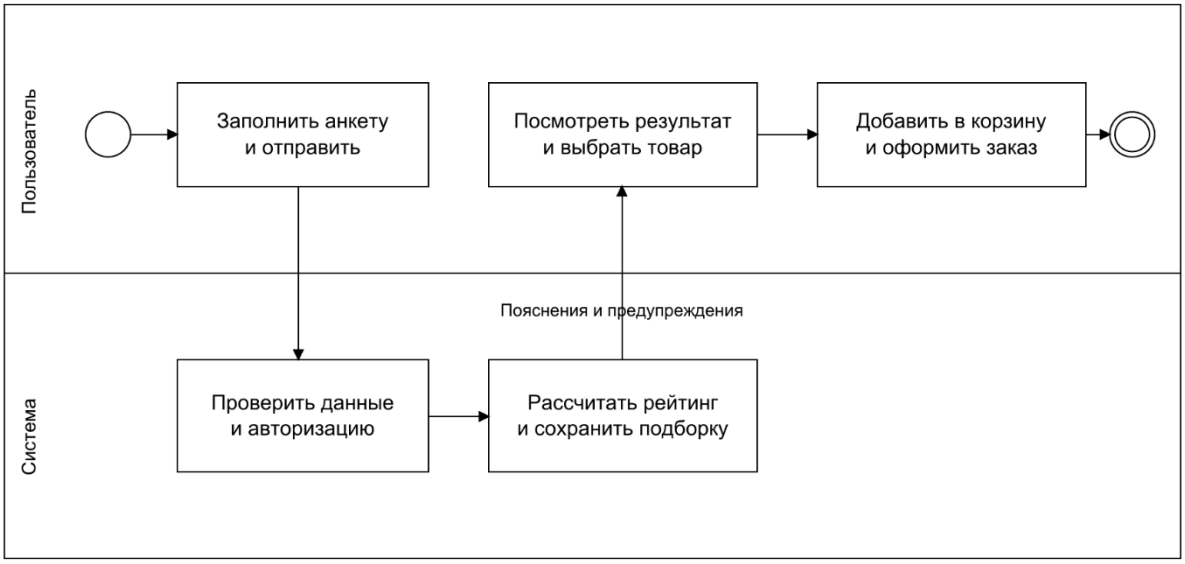


Рисунок 11 – «BPMN-диаграмма» пользовательского сценария[15]

2.3 УПРАВЛЕНИЕ ПРОЕКТОМ

2.3.1 Календарный план-график работ

План рассчитан на десять недель – со 2 сентября по 11 ноября 2026 года. Это план работ, а не журнал фактического выполнения. Дата на титульном

[15] Составлено автором

листе относится к защите учебной версии; оставшаяся часть графика рассматривается как план её доработки. Фактические даты завершения отдельных функций в работе отдельно не фиксировались.

– 1 неделя (2-8 сентября): настройка среды, установка «PostgreSQL», создание проекта, подключение «GitHub»;

– 2 неделя (9-15 сентября): создание моделей данных, контекста БД и первой миграции;

– 3 неделя (16-22 сентября): разработка главной страницы, навигации и базовых стилей;

– 4 неделя (23-29 сентября): разработка анкеты из 15 шагов и «JavaScript-логики»;

– 5 неделя (30 сентября - 6 октября): разработка алгоритма подбора, первые тесты;

– 6 неделя (7-13 октября): личный кабинет, сохранение подборок, избранное;

– 7 неделя (14-20 октября): регистрация, авторизация, корзина;

– 8 неделя (21–27 октября), план доработки: проверка PWA, каталога, поиска и пагинации;

– 9 неделя (28 октября – 3 ноября), план доработки: проверка административных функций и оформления заказов;

– 10 неделя (4–11 ноября), план доработки: расширение тестирования и актуализация документации.

Календарный план, включая последующую доработку, приведён на рисунке 12.

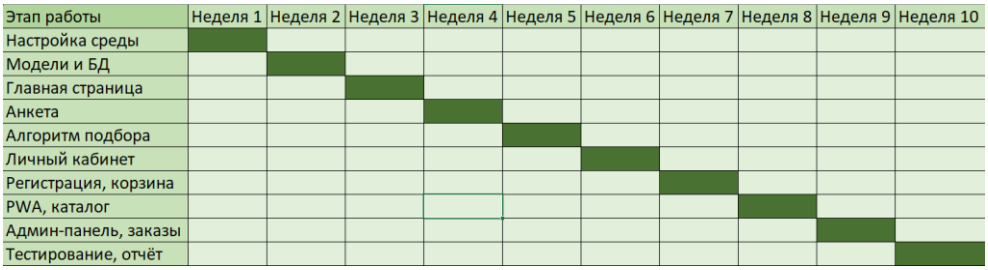


Рисунок 12 – План разработки и последующей доработки

2.3.2 Бюджет на разработку

Проект реализован без финансовых затрат на лицензии. Все инструменты – бесплатные: «Visual Studio Community», «PostgreSQL», «GitHub», «Bootstrap», «Font Awesome», «draw.io», «dbdiagram.io», «pgAdmin».

По моей оценке, разработка заняла 60–80 часов. Это оценка трудоёмкости, а не денежная стоимость. Для расчёта бюджета нужно дополнительно задать стоимость часа и учесть размещение, сопровождение и резервное копирование. Расчётный пример приведён в разделе 3.3.

2.3.3 Оценка рисков и способы их минимизации

В процессе разработки я столкнулась со следующими рисками, они приведены в таблице 7.

Таблица 7 – Риски проекта и способы их минимизации[16]

Риск	Что произошло	Как решено
Ошибка с паролем «PostgreSQL»	При «Update-Database» возникла ошибка «28P01: password authentication failed»	Найден «pg_hba.conf», метод изменён на trust, пароль сброшен через «ALTER USER», метод возвращён на «scram-sha-256»
Ошибка с типом «DateTime»	«PostgreSQL» требовал «UTC» для «timestamp with time zone»	Замена «DateTime.Now» на «DateTime.UtcNow»
Ошибка с «null-значениями»	При чтении «NULL» из БД возникало исключение	Все текстовые поля в «Profile» сделаны «nullable»
Неправильная работа алгоритма	Товары из «Ухода за телом» попадали в подборку для лица	Исключены лишние категории в «RecommendationService»
Проблемы с адаптивностью	На мобильных устройствах вёрстка ломалась	Использована «Bootstrap-сетка» и медиа-запросы
Дублирование логики	Методы «CheckGoal» и «CheckAllergies» были на 100+ строк	Переписаны через словари «goalRules» и «allergenMap»
«PWA» не устанавливался	Отсутствовали «manifest.json» и «service-worker.js»	Добавлены оба файла и иконки
Предупреждения компилятора	«Visual Studio» выдавал 57 предупреждений («CS8618, CS8602, CS8625, CS8604, CS8619»)	Добавлены = «null!»; для навигационных свойств, ?. и ?? «» для разыменования, «string?» для необязательных параметров

В таблице описаны проблемы, с которыми я столкнулась при разработке. Для последующих версий основные меры профилактики – проверка конфигурации подключения, единый подход к хранению дат и проверка необязательных полей. Отдельный риск алгоритма – компенсация штрафов бонусами; его следует учитывать при расширении тестирования.

[16] Составлено автором

3 РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ

3.1 ПРОГРАММНО-ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ РЕШЕНИЯ ПОСТАВЛЕННОЙ ЗАДАЧИ

3.1.1 Структура программного обеспечения

Приложение реализует структуру, описанную в разделе 2.2.1. Здесь я рассматриваю работу ключевых модулей: обращение к базе данных, расчёт рекомендаций и сохранение результата. Это позволяет связать проектную модель с конкретными методами программы.

3.1.2 Спецификации программных модулей

Ключевые программные модули системы можно разделить на три блока.

Первый – модуль доступа к базе данных. Класс «ApplicationDbContext» наследуется от «DbContext» («Entity Framework Core») и содержит по одному свойству «DbSet<T>» для каждой модели. Через него приложение взаимодействует с «PostgreSQL». Фрагмент класса «ApplicationDbContext» приведён в Приложении Г.

Главный метод RecommendationService.GetRecommendationsWithReasons() принимает Profile и возвращает RecommendationBundle. Он загружает товары, проверяет категории, вызывает 17 методов Check*() и формирует списки с пояснениями. Фрагмент приведён в приложении Д. Расчёт рейтинга и ограничения штрафной модели рассмотрены в разделе 2.2.2.

Третий – модуль обработки анкеты и подбора. Контроллер «QuizController» принимает данные анкеты, вызывает сервис подбора, сохраняет историю подборок и обновляет профиль пользователя. Фрагмент метода «Results()» контроллера «QuizController» показан в Приложении Е.

Перед вызовом сервиса проверяются авторизация и обязательные поля. После подбора в историю сохраняются доступные позиции основной подборки, затем создаётся или обновляется анкета. Запись рекомендаций и профиля выполняется двумя вызовами SaveChanges(); для их атомарного сохранения при дальнейшем развитии нужна общая транзакция.

Остальные модули системы:

– «HomeController» – отображение главной страницы;

– «QuizController» – анкета, подбор, корзина, избранное (уже упомянут выше);

– «AccountController» – регистрация, вход, выход, личный кабинет;

– «CatalogController» – каталог, поиск, фильтрация, пагинация;

– «CheckoutController» – оформление заказа и страница подтверждения;

– «GiftCardsController» – покупка и просмотр подарочных карт;

– «AdminController» – «CRUD»-операции с товарами.

3.1.3 Пользовательский интерфейс

Интерфейс системы построен на «HTML», «CSS» («Bootstrap 5.3») и «JavaScript». Все страницы адаптированы под мобильные устройства и работают в режиме «PWA». Основные экраны системы: главная страница, анкета, страница подборки, корзина, оформление заказа, личный кабинет и админ-панель [11].

Главная страница содержит приветствие, краткое описание системы, кнопку «Начать подбор» и навигационное меню со ссылками на каталог, корзину, личный кабинет, вход и регистрацию.

Анкета – центральный элемент системы. Она состоит из 15 шагов, они появляются по одному: после ответа на текущий шаг открывается следующий. Сверху расположен прогресс-бар, показывающий на каком шаге находится пользователь. Кнопка «Продолжить» переключает шаги, а на последнем меняется на «Получить подборку». Вся «JavaScript-логика» вынесена в отдельный файл «quiz.js». Внешний вид анкеты приведён на рисунке 13.

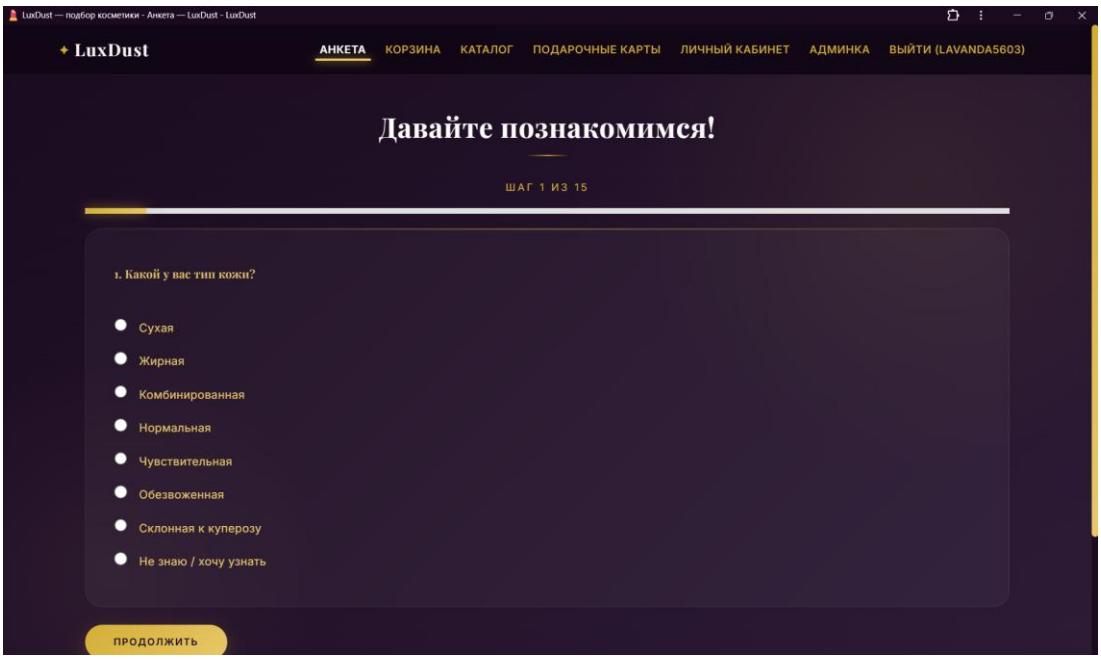


Рисунок 13 – Страница анкеты (первый шаг)[17]

Страница подборки отображает результат работы алгоритма. Пользователь видит ТОП-10 лучших товаров с фотографиями, названием, брендом, ценой и рейтингом совместимости. Под каждым товаром указаны причины, по которым он подошёл и предупреждения. Ниже расположен блок «Также может вам подойти» – товары, занявшие остальные места.

Корзина хранит выбранные товары. Добавление товаров в корзину реализовано через «AJAX» – без перезагрузки страницы. В корзине отображается список товаров с ценой и количеством, итоговая стоимость, а также кнопка «Оформить заказ». Удаление товаров выполняется с подтверждением. Внешний вид корзины приведён на рисунке 14.

[17] Составлено автором

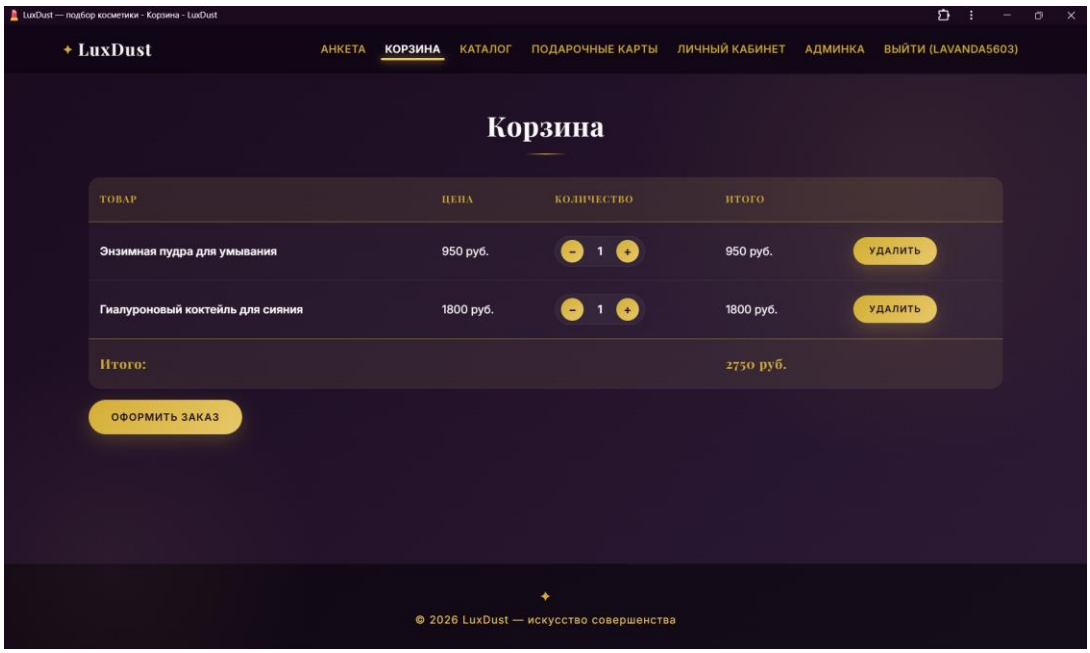


Рисунок 14 – Корзина с товарами[18]

Оформление заказа представлено страницей с формой, где пользователь выбирает способ доставки, дату доставки, время доставки, а также указывает телефон и комментарий. При выборе самовывоза поле «Адрес» скрывается с помощью «JavaScript». Внешний вид страницы оформления заказа приведён на рисунке 15.

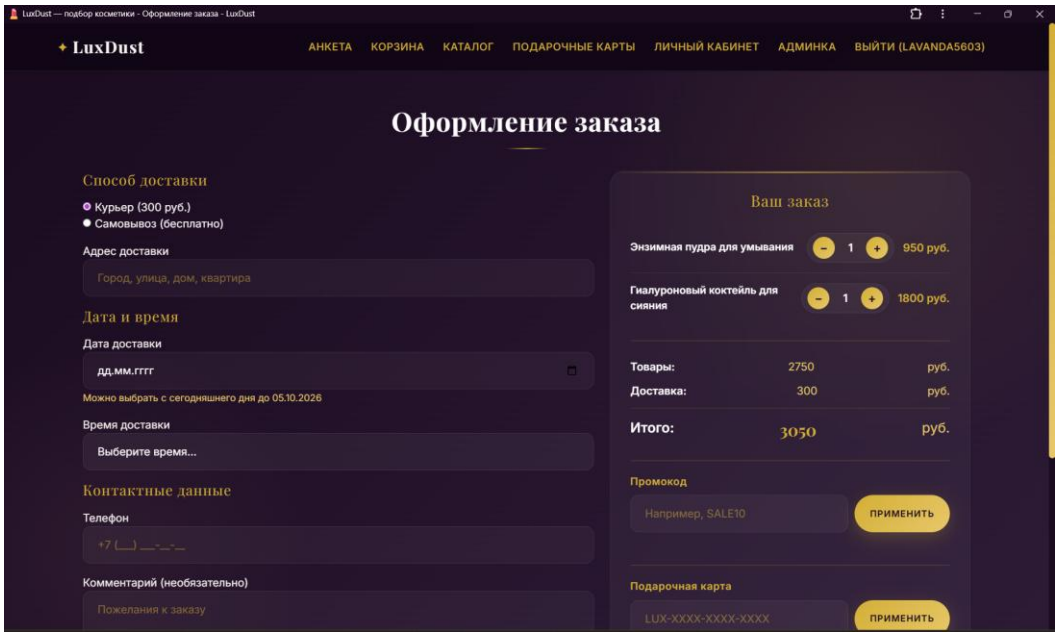


Рисунок 15 – Страница оформления заказа[19]

[18] Составлено автором
[19] Составлено автором

Страница «Спасибо за заказ» отображается после оформления. Она содержит номер заказа, способ доставки, дату и время, итоговую сумму и текущий статус. Статус заказа обновляется автоматически в зависимости от даты доставки и способа получения. Внешний вид страницы приведён на рисунке 16.

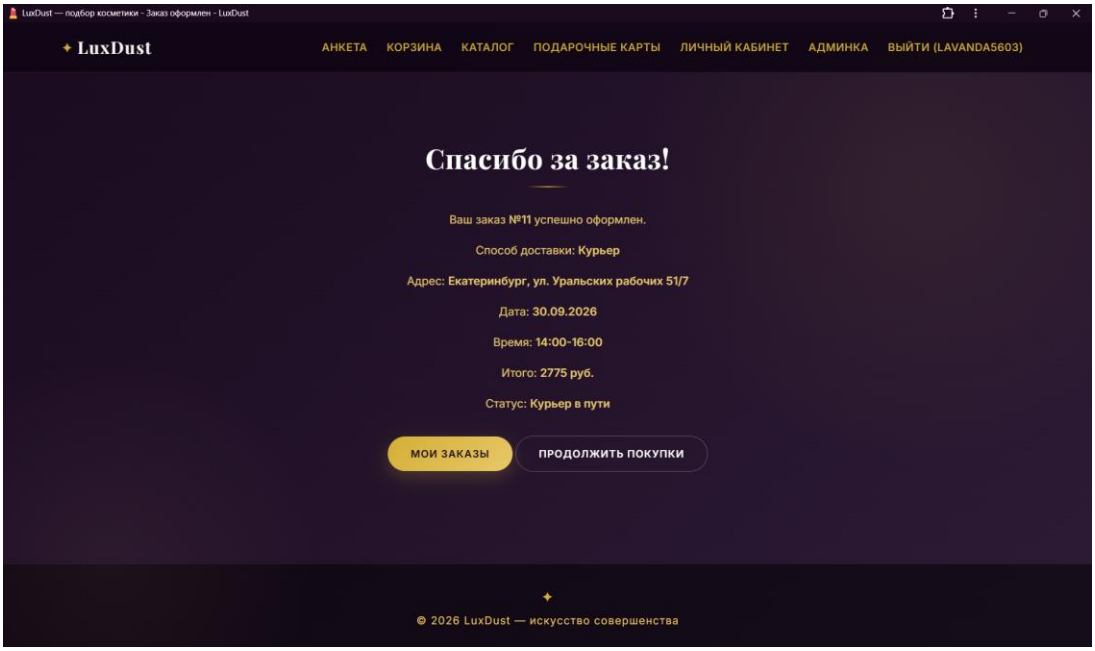


Рисунок 16 – Страница «Спасибо за заказ»[20]

Личный кабинет содержит данные анкеты пользователя с возможностью редактирования, историю подборок (с датами и составом), раздел «Избранное» и историю заказов.

Админ-панель доступна только администратору. Она содержит статистику системы (всего товаров, пользователей, заказов) и таблицу всех товаров с возможностью редактирования и удаления.

Последовательность вызовов при подборе показана на рисунке 17. QuizController передаёт анкету сервису, получает RecommendationBundle, сохраняет историю и возвращает представление. Обращение к PostgreSQL выполняется через ApplicationDbContext.

[20] Составлено автором

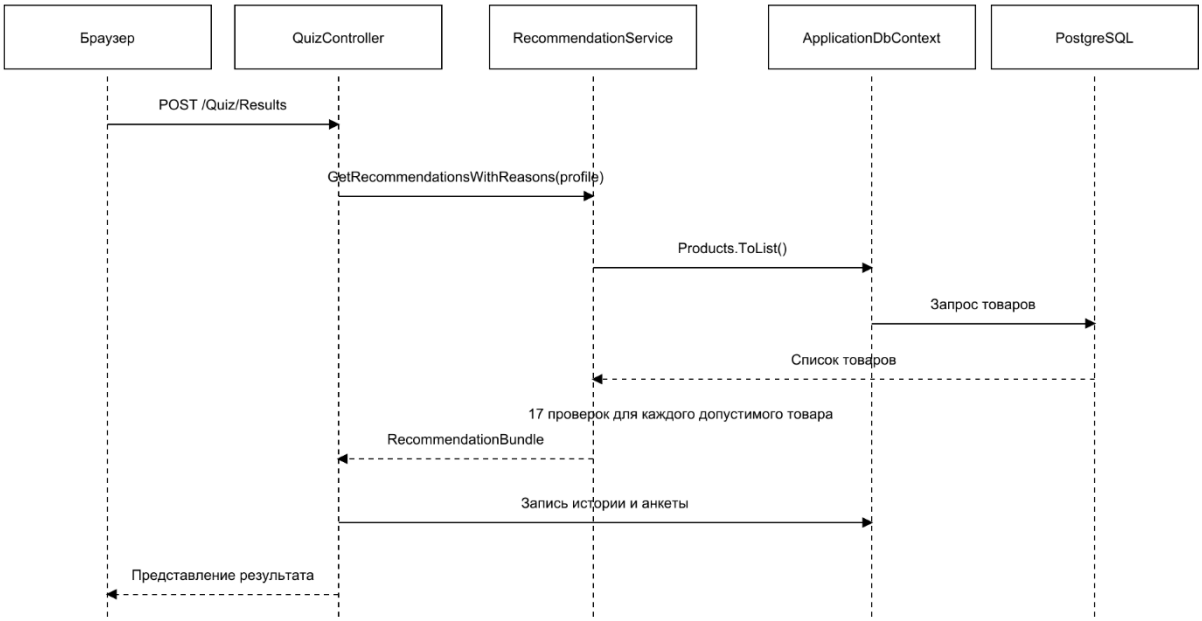


Рисунок 17 – Диаграмма последовательности «UML»[21]

3.1.4 Обеспечение информационной безопасности

Информационная безопасность системы обеспечивается на нескольких уровнях: аутентификация, хеширование паролей, разграничение прав доступа, защита персональных данных и валидация входных данных.

Используется «Cookie-аутентификация» «ASP.NET Core». После успешного входа пользователя сервер создаёт зашифрованный «Cookie», который хранится в браузере. При каждом запросе «Cookie» автоматически отправляется на сервер, и система определяет, кто именно обратился. Если пользователь не авторизован, доступ к защищённым страницам (анкета, личный кабинет, корзина) блокируется – происходит перенаправление на страницу входа.

Пароли не хранятся в открытом виде. При регистрации BCrypt.Net-Next создаёт хеш с солью, который записывается в базу данных. При входе метод BCrypt.Verify проверяет введённый пароль по сохранённому хешу. Хеширование не является обратимым шифрованием; стойкость также зависит от сложности пароля [6].

[21] Составлено автором

Предусмотрены три роли: гость, пользователь и администратор. Каждая роль имеет свой набор доступных функций. Распределение прав доступа приведено в таблице 8.

Таблица 8 – Распределение прав доступа[22]

Функция	Гость	Пользователь	Администратор
Просмотр каталога	Да	Да	Да
Просмотр карточки товара	Да	Да	Да
Прохождение анкеты	Нет	Да	Да
Получение подборки	Нет	Да	Да
Добавление в корзину	Нет	Да	Да
Добавление в избранное	Нет	Да	Да
Оформление заказа	Нет	Да	Да
Просмотр личного кабинета	Нет	Да	Да
Просмотр истории подборок	Нет	Да	Да
Редактирование анкеты	Нет	Да	Да
Админ-панель	Нет	Нет	Да
Добавление товара	Нет	Нет	Да
Редактирование товара	Нет	Нет	Да
Удаление товара	Нет	Нет	Да
Просмотр статистики	Нет	Нет	Да

Система хранит имя, email, хеш пароля, ответы анкеты и данные заказа. В учебной версии реализованы отдельные технические меры защиты. Перед использованием с реальными клиентами необходимо оформить правила обработки данных, согласие пользователя, сроки хранения и порядок удаления. Наличие HTTPS и хеширования само по себе не подтверждает выполнение всех требований к обработке персональных данных.

При регистрации проверяются формат и уникальность email, длина пароля не менее шести символов, наличие букв, цифры и специального символа, а также совпадение паролей. Эти правила заданы в RegisterViewModel и контроллере. Работа с базой через Entity Framework Core отделяет значения параметров от команд; проверка входных данных дополняет этот механизм, но не заменяет проверку прав доступа [10].

[22] Составлено автором

В Program.cs предусмотрено перенаправление на HTTPS. При развёртывании нужно настроить сертификат и проверить защищённое соединение. До такой проверки нельзя считать защиту передаваемых данных подтверждённой для выбранного сервера.

3.2 ТЕСТИРОВАНИЕ, ОЦЕНКА КАЧЕСТВА И ВНЕДРЕНИЕ

3.2.1 Исходные данные и результаты работы системы

В исходной проверке приложения я использовала пять сценариев с разными анкетами. Их входные данные и полученные товары сведены в таблицу 9. Проверки показывают выполнение отдельных сценариев, но не доказывают качество всех возможных подборок.

В первых двух сценариях проверялся подбор для сухой и жирной кожи при разных ограничениях бюджета и нежелательных компонентах. В третьем – обработка запроса на антивозрастной уход. Названия и цены относятся к записям учебного каталога и не подтверждают состав или свойства реальных товаров соответствующих брендов.

В четвёртом сценарии объединены чувствительная кожа, купероз и ограничение по эфирным маслам. В пятом задано несколько проблем и нежелательных компонентов. Для таких сочетаний важно проверять не только названия в подборке, но и итоговые баллы, предупреждения и заполненность признаков товара.

В пятом сценарии в результат попал товар за 2100 рублей при выбранном диапазоне 3000–10000 рублей. Это соответствует текущей реализации CheckBudget(): более дешёвый товар получает бонус как доступный вариант. Значит, бюджет трактуется как предпочтение и верхний ориентир, а не как строгий ценовой интервал.

Проверка исходного кода дополнительно показала ограничение: аллергены и несовпадения типа кожи обрабатываются штрафами. Наличие пяти удачных сценариев не исключает попадания конфликтного товара в

другой подборке. Для таких условий необходимы отдельные граничные проверки.

В следующий набор проверок следует включить пустой каталог, отсутствие обязательных ответов, совпадение одного и нескольких аллергенов, равные рейтинги товаров и попытку доступа обычного пользователя к административным действиям. Эти случаи обозначены как дальнейшая проверка, а не как уже полученные результаты.

Сводные результаты тестирования приведены в таблице 9.

Таблица 9 – Результаты тестирования[23]

№	Профиль	ТОП-3 товара	Ошибки
1	Сухая кожа, 30-40, 1000-3000 руб., аллергия на отдушки	Гиалуроновый коктейль для сияния («Librederm», 1800 руб.); Крем-суфле для сухой кожи («La Roche-Posay», 2400 руб.); Увлажняющий тоник с гиалуроновой кислотой («La Roche-Posay», 1100 руб.)	нет
2	Жирная кожа, до 20, до 1000 руб., аллергия на спирт	Гель для умывания с цинком («CeraVe», 900 руб.); Гель для умывания с салициловой кислотой («CeraVe», 950 руб.); Гель для умывания с чайным деревом («Avene», 850 руб.)	нет
3	Нормальная кожа, 50+, 3000-10000 руб., антивозрастной уход	Антивозрастной крем с коллагеном («Bioderma», 3200 руб.); Сыворотка с коллагеном («Librederm», 2700 руб.); Сыворотка с пептидами для упругости («Librederm», 2900 руб.)	нет
4	Чувствительная кожа, 20-30, 1000-3000 руб., купероз, вегетарианство	Сыворотка с центеллой для восстановления («Avene», 2300 руб.); Сыворотка с пробиотиками («CeraVe», 2500 руб.); Крем для лица с центеллой («La Roche-Posay», 2300 руб.)	нет
5	Комбинированная кожа, 30-40, 3000-10000 руб., много проблем и аллергий	Сыворотка с растительными стволовыми клетками («Bioderma», 3500 руб.); Сыворотка с цинком для проблемной кожи («CeraVe», 2100 руб.); Ночная сыворотка с ретинолом («Bioderma», 3200 руб.)	нет

[23] Составлено автором

3.2.2 Поэтапное описание работы с системой

Работа пользователя с системой выглядит следующим образом. Сначала он открывает главную страницу, нажимает «Регистрация», заполняет форму (имя, «email», пароль, подтверждение пароля) – система проверяет «email» на уникальность, пароль на сложность и сохраняет аккаунт. После входа пользователь нажимает «Начать подбор», открывается первый шаг анкеты, он отвечает на 15 вопросов, переключаясь кнопкой «Продолжить»; на последнем шаге кнопка меняется на «Получить подборку».

После отправки анкеты открывается подборка с баллами и пояснениями. Если есть предупреждения, их нужно учитывать при просмотре товара. Кнопки позволяют добавить позицию в корзину или избранное без полной перезагрузки страницы.

Перейдя в корзину, пользователь нажимает «Оформить заказ», выбирает способ доставки, дату и время, указывает телефон и подтверждает заказ. В завершение он может вернуться в личный кабинет, посмотреть историю подборок, избранное и историю заказов, а также отредактировать анкету.

Система поддерживает установку на телефон как «PWA». На рисунке 18 показана иконка приложения на домашнем экране телефона. На рисунке 19 показан полноэкранный режим работы приложения – без браузерной строки, как полноценное мобильное приложение [11].



Рисунок 18 – «PWA» на домашнем экране телефона[24]

[24] Составлено автором

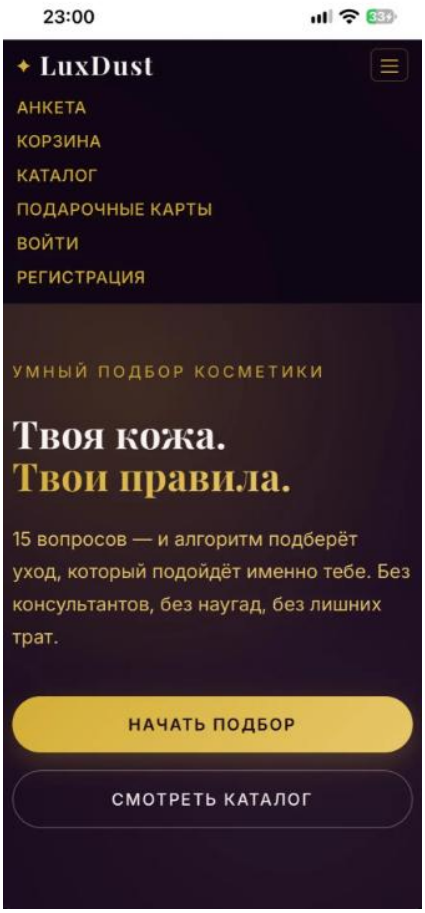


Рисунок 19 – «PWA» в полноэкранном режиме[25]

3.2.3 Порядок внедрения проекта

Для внедрения системы я определила следующий порядок. Сначала нужно подготовить сервер Windows или Linux со средой выполнения ASP.NET Core 8 и PostgreSQL. Ориентир для учебной установки – 2 ГБ оперативной памяти и 10 ГБ диска; достаточность ресурсов проверяется нагрузочным тестом. Далее создаётся отдельная учётная запись базы данных с необходимыми правами, настраивается подключение и публикуется приложение командой dotnet publish. Миграции применяются до запуска рабочей версии из подготовленной среды с SDK и инструментом dotnet-ef либо отдельным пакетом миграций.

Далее нужно настроить HTTPS, проверить подключение и основные сценарии, создать резервную копию и проверить её восстановление. Секреты

[25] Составлено автором

подключения следует передавать через настройки окружения. В завершение администратор знакомится с заполнением каталога и обработкой заказов. Успешная локальная демонстрация не заменяет проверку на сервере.

3.3 ОЦЕНКА ЭФФЕКТИВНОСТИ ПРОЕКТА

3.3.1 Анализ затрат на ресурсное обеспечение

При разработке я использовала бесплатные инструменты и не приобретала отдельные лицензии. Это сокращает прямые расходы учебного проекта. Однако труд разработчика, размещение приложения и сопровождение имеют стоимость, даже если при выполнении курсовой они не оплачивались отдельно.

Трудоёмкость оценивается в 60–80 часов. Для расчётного примера принимаю условную стоимость часа 500 рублей. Тогда стоимость разработки составляет 30–40 тыс. рублей. Ставка выбрана как допущение для расчёта и не представляется как установленная рыночная цена. Хостинг, домен и сопровождение в эту сумму не включены.

Для оценки возможной экономии бюджета после внедрения можно использовать формулу (2). В текущей работе нет наблюдений расходов пользователей до и после применения системы, поэтому численное значение экономии по этой формуле не определяется.

$$\text{Экономия} = \text{Бюджет}_{\text{было}} - \text{Бюджет}_{\text{стало}} \qquad (2)$$

где Бюджетбыло – расходы пользователя за выбранный период до применения системы; Бюджетстало – расходы за сопоставимый период после её применения. Для сравнения необходимо учитывать состав покупок и изменение цен.

3.3.2 Качественные и количественные эффекты

Для пользователя ожидаемый эффект состоит в сокращении повторного ввода условий и автоматическом сравнении товаров. Количественно проверить

его можно по времени выполнения одинакового задания вручную и с приложением. Время нужно измерять на группе пользователей, а качество подбора оценивать отдельно от скорости.

Для магазина возможны повторные обращения пользователей и более удобная работа с каталогом. Рост продаж и снижение возвратов зависят от ассортимента, трафика и других условий. В этой работе данные действующего магазина не собирались, поэтому проценты изменения этих показателей не рассчитываются.

Эргономический результат разработки – пошаговая анкета, пояснения к рейтингу и адаптивное отображение страниц. Удобство этих решений следует проверить наблюдением за пользователями: фиксировать ошибки, затруднения и время прохождения анкеты.

Доступ через браузер и PWA позволяет использовать один интерфейс на компьютере и телефоне. Эта возможность расширяет способы доступа к системе, но сама по себе не доказывает повышение качества рекомендаций.

3.3.3 Расчёт экономической эффективности

В таблице 10 сведены оценка трудозатрат и показатели, для которых ещё требуется собрать данные. Денежный результат и срок окупаемости нельзя определить без расходов на внедрение и измеренного эффекта.

Таблица 10 – Оценка затрат и условий эффективности

Показатель	Оценка и основание
Трудоёмкость	60–80 часов, оценка автора
Лицензии	Отдельные лицензии не приобретались
Условная ставка	500 руб./ч, допущение для расчёта
Стоимость разработки	30–40 тыс. руб., без размещения и сопровождения
Экономия пользователя	Не измерялась; требуется сравнение до и после
Изменение продаж и возвратов	Не измерялось; требуется опытное внедрение
Срок окупаемости	Не определён: нет измеренного денежного эффекта

Вывод. Проект выполнен без покупки лицензий, но имеет трудовые затраты. Расчётная стоимость разработки при принятой условной ставке составляет 30–40 тыс. рублей. Экономия времени и денег, изменение продаж и возвратов остаются проверяемыми предположениями; для их оценки необходимо опытное внедрение.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы я разработала информационную систему для подбора косметических средств. В приложении реализован путь от заполнения анкеты до просмотра рекомендаций и оформления заказа.

В первой главе я определила задачи автоматизации: собрать предпочтения пользователя, сравнить их с характеристиками товаров и сохранить результат. Рассмотрение существующих подходов помогло выбрать правила начисления баллов с понятными пояснениями.

В проектной части я описала архитектуру ASP.NET Core MVC и базу данных с 11 таблицами. Логику подбора вынесла в отдельный сервис. Рейтинг формируется по параметрам анкеты, категории и бонусам товара; он отражает правила программы, а не гарантированную пригодность косметического средства.

В ходе разработки реализовала все ключевые функции: регистрацию и авторизацию с хешированием паролей, анкету из 15 шагов, алгоритм подбора, личный кабинет с историей подборок и избранным, корзину с «AJAX-добавлением», оформление заказа с промокодами и подарочными картами, а также админ-панель для управления каталогом. Система поддерживает «PWA»: её можно установить на телефон и пользоваться как полноценным мобильным приложением.

Тестирование я описала на пяти сценариях из учебного каталога. Они показывают работу основных вариантов подбора. При проверке кода выявлены ограничения штрафной модели: конфликтный товар может сохранить высокий итоговый рейтинг. Для следующей версии я выделила строгую фильтрацию нежелательных компонентов и дополнительные граничные проверки.

В оценке затрат я учла 60–80 часов разработки. При условной ставке 500 рублей в час это составляет 30–40 тыс. рублей без размещения и сопровождения. Подтвердить экономию бюджета пользователя и влияние на продажи можно только после сбора данных при внедрении.

Практический результат – учебное приложение с анкетой, каталогом, историей рекомендаций и административным интерфейсом. Перед использованием с реальными клиентами требуется проверить полноту данных о товарах, защиту персональных данных и ограничения алгоритма.

Дальнейшее развитие я связываю с проверкой весов на пользовательских оценках, строгим исключением товаров с нежелательными компонентами, контролем целостности базы данных и расширением тестирования. После этого можно оценивать рекомендации на более широком наборе анкет.

Цель разработки учебной версии достигнута: система формирует подборку по анкете и сохраняет результат. При этом её эффективность для реального покупателя остаётся предметом дальнейшей проверки.

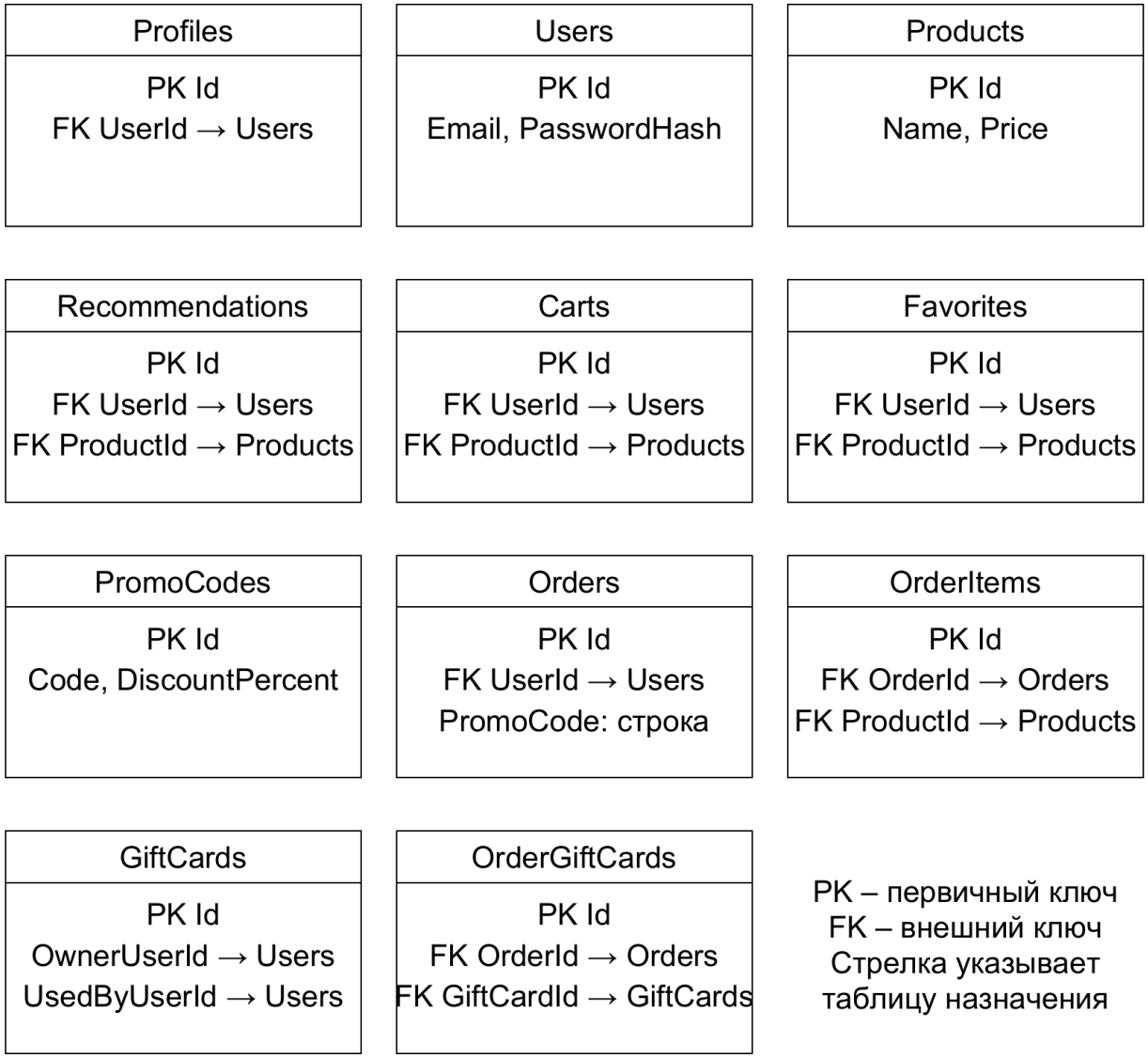
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Выручка российских производителей косметики в 2025 году превысила 1,16 трлн руб., на их долю пришлось до 80% оборота // Финмаркет. – 2026. – 3 мая. – URL: https://www.finmarket.ru/news/6613595 (дата обращения: 24.10.2026).
2. Гусенко М., Российская крем-революция: как отечественная косметика захватила рынок // Российская газета. – 2026. – 3 мая. – URL: https://rg.ru/2026/05/03/rossijskaia-krem-revoliuciia-kak-otechestvennaia-kosmetika-zahvatila-80-rynka.html.
3. Морозова Я., Россияне начали экономить на косметике // Retail.ru. – 2026. – 26 февраля. – URL: https://www.retail.ru/news/rossiyane-nachali-ekonomit-na-kosmetike/ (дата обращения: 24.10.2026).
4. A Novel Approach for Smart Skincare Solution: Derma AI Recommendation System // IEEE Xplore. – 2025. – DOI: 10.1109/ETCC65847.2025.11108513.
5. A Review on Cosmetic Product Recommendation Using Deep Learning // IEEE Xplore. – 2024. – DOI: 10.1109/ICSCSA64454.2024.00009.
6. BCrypt.Net-Next // NuGet Gallery. – URL: https://www.nuget.org/packages/BCrypt.Net-Next/ (дата обращения: 24.10.2026).
7. Deep Learning-Based Hybrid Recommendation System for Safe Approach to Curated Cosmetics // IEEE Xplore. – 2025. – DOI: 10.1109/IDCIOT64235.2025.10915032.
8. Generative AI-based Explainable Personalized Hybrid Skincare Recommendation System // Information Processing Society of Korea Transactions on Software and Data Engineering. – 2025. – Vol. 14, no. 9. – P. 695–703. – DOI: 10.3745/TKIPS.2025.14.9.695. – URL: https://www.kci.go.kr/kciportal/landing/article.kci?arti_id=ART003251754 (дата обращения: 24.10.2026).

9. Microsoft. Overview of ASP.NET Core MVC // Microsoft Learn. – URL: https://learn.microsoft.com/aspnet/core/mvc/overview?view=aspnetcore-8.0 (дата обращения: 24.10.2026).
10. Microsoft. Обзор Entity Framework Core // Microsoft Learn. – URL: https://learn.microsoft.com/ru-ru/ef/core/ (дата обращения: 24.10.2026).
11. Microsoft. Get started developing a PWA // Microsoft Edge Developer documentation. – URL: https://learn.microsoft.com/en-us/microsoft-edge/progressive-web-apps-chromium/how-to/ (дата обращения: 24.10.2026).
12. PostgreSQL Global Development Group. PostgreSQL 16 Documentation // PostgreSQL. – URL: https://www.postgresql.org/docs/16/index.html (дата обращения: 24.10.2026).
13. Skincare Recommendation System based on User's Skin Characteristics Similarity using Collaborative Filtering Approach // IEEE Xplore. – 2024. – DOI: 10.1109/ICEECIT63698.2024.10859798.

ПРИЛОЖЕНИЕ А

Схема таблиц базы данных и ссылок между ними



ПРИЛОЖЕНИЕ Б

SQL-описание таблицы Products по текущей модели

```
CREATE TABLE "Products" (
    "Id" integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    "Name" text,
    "Brand" text,
    "Category" text,
    "SubCategory" text,
    "Price" integer NOT NULL,
    "SkinType" text,
    "Problem" text,
    "AllergenFree" boolean NOT NULL,
    "Season" text,
    "ImageUrl" text,
    "Description" text,
    "Rating" double precision NOT NULL,
    "IsNew" boolean NOT NULL,
    "IsOnSale" boolean NOT NULL,
    "CreatedAt" timestamp with time zone NOT NULL,
    "SunSensitivity" boolean NOT NULL,
    "TendencyToEdema" boolean NOT NULL,
    "TexturePreference" text,
    "HasProfessionalCare" boolean NOT NULL,
    "ReadyForMultiStep" boolean NOT NULL,
    "HasParabens" boolean NOT NULL,
    "HasFragrance" boolean NOT NULL,
    "HasAlcohol" boolean NOT NULL,
    "HasSilicones" boolean NOT NULL,
    "HasSulfates" boolean NOT NULL,
    "HasGluten" boolean NOT NULL,
    "HasNuts" boolean NOT NULL,
    "HasEssentialOils" boolean NOT NULL
);
```

ПРИЛОЖЕНИЕ В

Фрагмент метода «CheckAllergies()» обработки аллергий

```
var allergenMap = new Dictionary<string, Func<Product, bool>>
{
   { "Парабены", p => p.HasParabens },
   { "Отдушки", p => p.HasFragrance },
   { "Спирт", p => p.HasAlcohol },
   { "Силиконы", p => p.HasSilicones },
   // ...
};
foreach (var allergy in userAllergies)
{
   if (allergenMap.ContainsKey(allergy) && allergenMap[allergy](product))
   {
      res.Score -= 50;
      warnings.Add($"содержит «{allergy}» — аллергия!");
   }
   else
   {
      res.Score += 10;
   }
}
```

Фрагмент класса «ApplicationDbContext»

```
using Microsoft.EntityFrameworkCore;
using LuxDustApp.Models;
namespace LuxDustApp.Data
{
   public class ApplicationDbContext : DbContext
   {
      public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
         : base(options)
      {
      }
      public DbSet<User> Users { get; set; }
      public DbSet<Profile> Profiles { get; set; }
      public DbSet<Product> Products { get; set; }
      public DbSet<Recommendation> Recommendations { get; set; }
      public DbSet<Favorite> Favorites { get; set; }
      public DbSet<Cart> Carts { get; set; }
      public DbSet<PromoCode> PromoCodes { get; set; }
      public DbSet<GiftCard> GiftCards { get; set; }
      public DbSet<OrderGiftCard> OrderGiftCards { get; set; }
      public DbSet<Order> Orders { get; set; }
      public DbSet<OrderItem> OrderItems { get; set; }
   }
}
```

Фрагмент метода «GetRecommendationsWithReasons()»

```
public RecommendationBundle GetRecommendationsWithReasons(Profile profile)
{
   var allProducts = _context.Products.ToList();
   var result = new Dictionary<Product, RecommendationResult>();
   var bundle = new RecommendationBundle();
   if (allProducts == null || !allProducts.Any())
      return bundle;
   var excludedCategories = new List<string> { "Подборки", "Уборка и стирка", "Хобби и
творчество", "Фигура мечты" };
   bool userWantsFaceCare = !string.IsNullOrEmpty(profile.Goal) &&
      RecommendationConstants.FaceCareGoals.Any(g => profile.Goal.Contains(g));
   var userAllergies = (profile.Allergies ?? "")
      .Split(',', StringSplitOptions.RemoveEmptyEntries)
      .Select(a => a.Trim())
      .Where(a => a != "Нет")
      .ToList();
   foreach (var product in allProducts)
   {
      if (!string.IsNullOrEmpty(product.Category) &&
excludedCategories.Contains(product.Category))
         continue;
      if (userWantsFaceCare && !string.IsNullOrEmpty(product.Category) &&
         RecommendationConstants.NonFaceCareCategories.Contains(product.Category))
         continue;
      var res = new RecommendationResult();
      var reasons = new List<string>();
      var warnings = new List<string>();
      CheckSkinType(profile, product, res, reasons, warnings);
      CheckAge(profile, product, res, reasons, warnings);
      CheckBudget(profile, product, res, reasons, warnings);
      CheckProblems(profile, product, res, reasons, warnings);
      CheckAllergies(profile, product, userAllergies, res, reasons, warnings);
      CheckSeason(profile, product, res, reasons, warnings);
      CheckBrands(profile, product, res, reasons, warnings);
      CheckGoal(profile, product, res, reasons, warnings);
      CheckStress(profile, product, res, reasons, warnings);
      CheckDiet(profile, product, res, reasons, warnings);
      CheckSunSensitivity(profile, product, res, reasons, warnings);
      CheckEdema(profile, product, res, reasons, warnings);
      CheckProfessionalCare(profile, product, res, reasons, warnings);
      CheckTexture(profile, product, res, reasons, warnings);
      CheckMultiStep(profile, product, res, reasons, warnings);
      CheckCategory(profile, product, res, reasons, warnings, userWantsFaceCare);
      CheckBonuses(product, res, reasons);
      res.Reasons = reasons.Any() ? " ✓ " + string.Join("; ", reasons) + "." : "Нет явных
совпадений.";
      if (warnings.Any())
         res.Reasons += " ! " + string.Join("; ", warnings) + ".";
```

```
        result[product] = res;
    }
    var sorted = result.OrderByDescending(kv => kv.Value.Score).ToList();
    bundle.Top = sorted.Take(10).Select(kv => kv.Key).ToList();
    bundle.TopReasons = sorted.Take(10).ToDictionary(kv => kv.Key, kv => kv.Value);
    bundle.Middle = sorted.Skip(10).Take(50).Select(kv => kv.Key).ToList();
    bundle.MiddleReasons = sorted.Skip(10).Take(50).ToDictionary(kv => kv.Key, kv =>
kv.Value);
    return bundle;
}
```

Фрагмент метода «Results()» контроллера «QuizController»

```
[HttpPost]
public IActionResult Results(Profile profile)
{
   var userId = GetUserId();
   if (userId == null) return RedirectToAction("Login", "Account");
   if (string.IsNullOrEmpty(profile.SkinType) || profile.Age <= 0 || profile.Budget <= 0)
   {
      ModelState.AddModelError("", "Пожалуйста, заполните все обязательные поля
анкеты.");
      return View("Step1", profile);
   }
   var bundle = _recommendationService.GetRecommendationsWithReasons(profile);
   foreach (var kvp in bundle.TopReasons)
   {
      _context.Recommendations.Add(new Recommendation
      {
         UserId = userId.Value,
         ProductId = kvp.Key.Id,
         Score = kvp.Value.Score,
         Reasons = kvp.Value.Reasons,
         RecommendedAt = System.DateTime.UtcNow
      });
   }
   _context.SaveChanges();
   var existingProfile = _context.Profiles.FirstOrDefault(p => p.UserId == userId.Value);
   if (existingProfile == null)
   {
      profile.UserId = userId.Value;
      profile.UpdatedAt = System.DateTime.UtcNow;
      _context.Profiles.Add(profile);
   }
   else
   {
      existingProfile.SkinType = profile.SkinType;
      existingProfile.Age = profile.Age;
      existingProfile.Budget = profile.Budget;
      existingProfile.Problems = profile.Problems;
      existingProfile.Allergies = profile.Allergies;
      existingProfile.Season = profile.Season;
      existingProfile.FavoriteBrands = profile.FavoriteBrands;
      existingProfile.Goal = profile.Goal;
      existingProfile.StressLevel = profile.StressLevel;
      existingProfile.DietType = profile.DietType;
      existingProfile.SunSensitivity = profile.SunSensitivity;
      existingProfile.TendencyToEdema = profile.TendencyToEdema;
      existingProfile.HasProfessionalCare = profile.HasProfessionalCare;
      existingProfile.TexturePreference = profile.TexturePreference;
      existingProfile.ReadyForMultiStep = profile.ReadyForMultiStep;
```

```
        existingProfile.UpdatedAt = System.DateTime.UtcNow;
    }
    _context.SaveChanges();
    return View(bundle);
}
```